SK4 ELF Toolchain Reference

for Ezairo® 7100

M-20804-010
January 2015



ON Semiconductor®

© SCILLC, 2015
Previous Edition © 2014
"All Rights Reserved"

ON Semiconductor and  are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

Table of Contents

Page

1. Introduction . . . 7
    1.1 Purpose . . . 7
    1.2 SK4 Tools . . . 7
    1.3 Building Software . . . 8
        1.3.1 Building Software with the IDE . . . 8
        1.3.2 Building Software from the Command Line . . . 8
    1.4 Intended Audience . . . 8
    1.5 Conventions . . . 8
    1.6 Manual Organization . . . 9
    1.7 Further Reading . . . 9
2. Building Software with the IDE . . . 10
    2.1 Introduction . . . 10
    2.2 Build Stages . . . 10
    2.3 Build Settings . . . 10
    2.4 Build Variables . . . 12
    2.5 Auto-Build . . . 13
3. Running the Command-Line Tools . . . 14
    3.1 Introduction . . . 14
    3.2 The Stages of Compilation . . . 14
        3.2.1 Preprocessing. . . . 14
        3.2.2 Assembling . . . 14
        3.2.3 Linking. . . . 14
        3.2.4 Archiving/Librarian. . . . 14
    3.3 CFCC Usage . . . 15
        3.3.1 Syntax . . . 15
        3.3.2 CFCC - Toolchain Driver Options . . . 16
    3.4 Error Codes . . . 17
    3.5 Error Messages . . . 17
        3.5.1 “Option error(option): message” . . . 17
        3.5.2 “IO Error: message” . . . 17
        3.5.3 “No input files” . . . 18
        3.5.4 “Unknown target specified in -b: target” . . . 18
        3.5.5 “Cannot specify -o with -c and multiple compilations” . . . 18
        3.5.6 “Error: argument is not a file” . . . 18
        3.5.7 “Don’t know how to build file: filename” . . . 18
        3.5.8 “System exception when running tool” . . . 18
        3.5.9 “Error running tool: message” . . . 19
        3.5.10 “Unexpected error running tool” . . . 19
        3.5.11 “Error moving filename to destination: message” . . . 19
        3.5.12 “Error copying filename to destination: message” . . . 19
        3.5.13 “An unexpected error has occurred in tool. Please contact ON Semiconductor customer support” 19

4. Preprocessing Source Code 20
4.1 Introduction 20
4.2 Running the Preprocessor 20
4.2.1 Preprocessing Options 20
4.2.2 Predefined Macros 21
4.3 ON Semiconductor Preprocessing Language 22
4.3.1 Preprocessing Directive Summary 22
4.3.2 Preprocessing Procedure 23
4.3.3 Lexical Extensions 24
4.3.3.1 Dollar in Names 24
4.3.3.2 Comment Removal 24
4.3.3.3 Escaping the Hash Character 24
4.3.4 Preprocessor Expressions 24
4.3.5 Directive Extensions 25
4.3.5.1 Details of #include File Searching 25
4.3.5.2 Comment Removal 26
4.3.5.3 Generating a Warning with #warning 26
4.3.5.4 Repeating Text Blocks Using #rept 26
4.3.5.5 Defining Multi-Line Macros with #macro 26
4.3.5.6 Defining Numeric Macros with #set 27
4.3.5.7 Comparison of Macro Types 27
5. Assembling Programs 30
5.1 Introduction 30
5.2 Running the Assembler 30
5.2.1 Assembler Options 31
5.2.2 Instruction Restriction Checks 31
5.2.2.1 Restriction Categories 31
5.2.2.2 Restriction Error Messages 32
5.3 CFX Assembly Language 33
5.3.1 Lexical Conventions 34
5.3.2 Assembly Language Syntax 35
5.3.2.1 Instruction Forms 35
5.3.2.2 Synonyms and Synthesized Instructions 36
5.3.2.2.1 Synonym Instructions and Synthesized Instructions 36
5.3.2.2.2 Alternate Condition Forms 37
5.3.2.2.3 Addressing Mode Synonyms 37
5.3.2.3 Numeric Instruction Operand Syntax 37
5.3.3 Assembler Directives 39
5.3.3.1 Directives for Declaring Symbols 39
5.3.3.2 Directives for Defining Data 40
5.3.3.3 Directives for Changing the Program Counter 41
5.3.3.4 Directives for Partitioning Memory 41
5.3.3.5 Directives for Controlling Checking 43
5.3.3.6 Directives for Preprocessing 45
5.3.3.7 Directives for Controlling the Listing File 46

5.3.3.8 Directives for Defining Special Object File Entries . . . 47
5.4 Generating Debugging Symbols . . . 49
6. Using the Object Librarian . . . 50
6.1 Introduction . . . 50
6.2 Ordering Object Modules in a Library . . . 50
7. Linking Object Modules . . . 51
7.1 Introduction . . . 51
7.2 Linking Relocatable Object Code . . . 51
7.2.1 Linking Options . . . 51
7.3 Ordering Object Module Libraries . . . 53
7.4 Memory Configuration . . . 54
7.4.1 Memory Allocation for SK4 Systems . . . 54
7.5 Load Map Format . . . 55
7.6 Converting Binary Executables to Text Format . . . 55
8. Non-Volatile Memory Management . . . 57
8.1 Introduction . . . 57
8.2 Features . . . 57
8.2.1 Flexible Layout Arrangement . . . 57
8.2.2 Optimization Through Automatic Equivalent Section Detection . . . 57
8.2.3 Encryption Support . . . 58
8.2.4 EEPROM Image Upload . . . 58
8.2.5 Graphical Layout View Using NVI Manager . . . 58
8.3 Using NVMM . . . 58
8.3.1 Creating an EEPROM Image Using the Project Wizard . . . 58
8.3.1.1 NVMM Applications Settings . . . 58
8.3.1.2 Ezairo 7100 EEPROM Layout Defaults . . . 59
8.3.1.3 Editing Build Settings . . . 60
8.3.1.4 Adding an Application . . . 62
8.3.1.5 Securing an EEPROM Image . . . 64
8.3.2 Non-Volatile Memory Data Upload Through the IDE . . . 65
8.3.3 Reserving a Section of EEPROM . . . 66
8.3.4 Viewing an EEPROM Image with NVI Manager . . . 67
8.3.4.1 Virtual File System (VFS) Tree View . . . 67
8.3.4.2 Graphical EEPROM Layout View . . . 68
8.3.4.3 Additional NVI File Views . . . 69
8.3.4.4 NVI Manager Preferences . . . 69
8.4 Options . . . 69
8.4.1 General Options . . . 70
8.4.1.1 Bootloader-Specific Options . . . 71
8.4.1.2 Application-Specific Options . . . 71
8.4.2 Global Field Options . . . 73
8.4.3 Ezairo 7100 (SK4FS) Areas . . . 74
8.4.4 Loading Arbitrary Data Files . . . 74
9. Non-Volatile Image Tools . . . 76
9.1 Introduction . . . 76

9.2 Non-Volatile Memory Download Tool . . . 76
9.2.1 Using NVMD Within the IDE . . . 76
9.2.1.1 Downloading a .nvi File . . . 76
9.2.1.2 NVI Download Launch Configurations . . . 77
9.2.1.2.1 Download Tab . . . 79
9.2.1.2.2 Connection Tab . . . 79
9.2.2 Usage . . . 80
9.3 Non-Volatile Image Dump Tool . . . 82
A. File Formats . . . 84
A.1 Introduction . . . 84
A.2 Executable and Linking Format (ELF) . . . 84
A.3 Relocatable Object File Format . . . 84
A.4 Non-Volatile Memory Image Format (.nvi) . . . 84
A.4.1 NVI File Structure . . . 84
A.4.2 Headers . . . 84
A.4.2.1 Main File Header . . . 84
A.4.2.2 Generic Block Header . . . 85
A.4.3 String Table . . . 86
A.4.4 VFS Node Chunks . . . 86
A.4.5 Data Chunks . . . 87
A.4.6 CRC Values . . . 88

CHAPTER 1
Introduction

1.1 PURPOSE

This *SK4 ELF Toolchain Reference for Ezairo® 7100* provides reference information for building software with the Ezairo 7100 Evaluation and Development Kit (EDK). It describes the preprocessing, assembling, and linking processes, and how to do all three together using the Toolchain Driver. It also describes how to create and load non-volatile file system images. Furthermore, the manual provides detailed information on file formats.

The SK4 ELF Toolchain is the default toolchain for all Ezairo 7100 projects. It consists of the tools necessary to assemble code for CFX, HEAR, Filter Engine, and ARM® Cortex™-M3 cores. This toolchain is based on the open source GNU Binutils toolchain. Because this toolchain generates its output in the widely-used standard ELF file format, it is referred to here as the *SK4 ELF toolchain*.

The build process can be performed from a command line or through the Integrated Development Environment (IDE) as described in this introduction.

1.2 SK4 TOOLS

The SK4 ELF Toolchain consists of several tools that are involved in the build process. These tools are:

- CFCC Assembler: Creates object files and associated metadata files from CFX assembly code source files
- HEAR Assembler: Creates assembly header files and object files
- Filter Engine Assembler: Creates CFX assembly code source files from Filter Engine source files
- CFCC Linker: Creates executable files from object files
- GCC Compiler: Creates object files from ARM Cortex-M3 processor source files
- GCC Linker: Creates ARM Cortex-M3 processor executable files from ARM Cortex-M3 processor object files
- ELF2CFS: Converts ARM Cortex-M3 processor executable files into CFX assembly code which can be loaded through the CFX core
- Non-Volatile Memory Manager (NVMM): Creates an EEPROM image from a CFX executable

Aside from the ELF2CFS tool, you can modify all of the settings for these tools (which are options passed to the command line) from the build settings dialog. For more information about these tools, including details about their different options, see their respective reference manuals.

ELF2CFS is a special tool that allows you to program the ARM Cortex-M3 core through the CFX core. This tool is not accessible through the build settings dialog; instead, it is defined in the *makefile.targets* file.

IMPORTANT: To use the SK4 tools for building, you must specify the GNU toolchain option for the Toolchain Driver. For backwards compatibility, the command line also supports the Archelon toolchain. For files produced by that toolchain, the Ezairo 7100 EDK supports the following features:

- **Loading an executable to a chip**
- **Running and debugging**

This manual assumes you are using the SK4 ELF toolchain with the GNU toolchain option. For more information about the Archelon toolchain, see the Toolchain Reference manuals in SignaKlara EDK 5.2.1.

1.3 Building Software

The building process translates source code to machine code that can run directly on the chip. You can build software with the IDE or from the command line.

1.3.1 Building Software with the IDE

The build system in the IDE automatically generates build rules based on the contents of the project, and uses an integrated builder (based on the popular Ant build tool) to generate the output (*.cfe, .a*, etc.) files. The build settings are configured through a Settings Editor, and the build process can be extended with customized pre- and post-build steps. The IDE also has a feature which allows for the automatic building of projects whenever a file is saved. For more information about these features, see Chapter 2, “Building Software with the IDE” on page 10.

1.3.2 Building Software from the Command Line

The software tools that the IDE uses to build code are also available from the Ezairo 7100 Tools Command Prompt. The main tool is CFCC, the Toolchain Driver, which you can use to call most of the other tools. The building process includes the following sub-processes:

- Preprocessing
- Assembly
- Linking object modules

For an overview of how you can combine these processes to build software applications, see Chapter 3, “Running the Command-Line Tools” on page 14.

1.4 Intended Audience

This manual is intended for readers who have some familiarity with the C language. For more information on concepts such as the C preprocessor, C-style comments, C expression operators, C-style numeric literals, and C string escape sequences, see any of the standard works on the subject, such as *The C Programming Language* by Kernighan and Ritchie.[1]

1.5 Conventions

In general, numbers are presented in decimal notation. In cases where hexadecimal notation is more convenient, these numbers are identified by the prefix “0x”. For example, the decimal number 123456 can also be represented as 0x1E240.

The following special fonts are used in this manual to signify particular types of information:

`monospace font`	Commands and their options, error messages, code samples and code snippets.
`mono bold`	A placeholder that represents where you would specify the appropriate information. For example, you would replace **`filename`** with the actual name of the file.
`<angle brackets>`	In Chapter 5, “Assembling Programs” on page 30, the convention is to use angle brackets (<>) to indicate placeholders. For example, you would replace `<num-expr>` with a numeric expression.

1. Kernighan, Brian W., and Ritchie, Dennis M. The C Programming Language. 2nd ed. Prentice Hall, 1988.

italics File and path names, or any portion of them.

bold Used for menu names and menu items.

1.6 Manual Organization

The *SK4 ELF Toolchain Reference for Ezairo 7100* contains the following chapters and appendices.

- **Chapter 1: Introduction**, describes the purpose of this manual and how the book is organized, provides system requirements, and gives a list of suggested reading for more information.
- **Chapter 2: Building Software with the IDE**, describes the Integrated Development Environment features for building software.
- **Chapter 3: Running the Command-Line Tools**, provides a high-level overview of the software development process and how to use CFCC.
- **Chapter 4: Preprocessing Source Code**, discusses issues related to the preprocessing stage of software compilation.
- **Chapter 5: Assembling Programs**, provides an overview of assembly language programming with the EDK.
- **Chapter 6: Using the Object Librarian**, provides information on creating and manipulating libraries of object modules.
- **Chapter 7: Linking Object Modules**, discusses issues related to the linking stage of software compilation.
- **Chapter 8: Non-Volatile Memory Management**, explains how to use the Non-Volatile Memory Manager (NVMM) to create loadable, non-volatile images for Ezairo 7100.
- **Chapter 9: Non-Volatile Image Tools**, describes loading non-volatile image files to the Ezairo 7100 non-volatile memory.
- **Appendix A: File Formats**, discusses the file formats used by the assembler toolchain, and the file format for storing non-volatile memory images once the layout has been processed.

1.7 Further Reading

For more information, refer to the following documents:

- *Integrated Development Environment User's Guide*
- *Hardware Reference Manual for Ezairo 7100*
- *CFX DSP Architecture Manual*
- *HEAR Configurable Accelerator Reference Manual*
- *Filter Engine Reference Manual*
- *ARM Cortex-M3 Processor Toolchain Reference*
- *Evaluation and Development Board Manual for Ezairo 7100*
- *Firmware Reference Manual for Ezairo 7100*
- *Eclipse Workbench User Guide*
- Kernighan, Brian W., and Ritchie, Dennis M. *The C Programming Language*. 2nd ed. (Prentice Hall, 1988)
- Ant documentation: http://ant.apache.org/manual/
- The Jython Web site: http://www.jython.org/
- The Python Web site: http://python.org/
- Lutz, Mark, and Ascher, David. *Learning Python*. 2nd ed. (O'Reilly, 2003)
- Lutz, Mark. *Programming Python*. 2nd ed. (O'Reilly, 2001)
- Pedroni, Samuele, and Rappin, Noel. *Jython Essentials*. 1st ed. (O'Reilly, 2002)

CHAPTER 2
Building Software with the IDE

2.1 Introduction

The following sections describe how to build software in the Integrated Development Environment (IDE). Using this interface, you can configure the build settings for your project, add your own custom build steps, and even have the build occur automatically when a source file changes.

The project build system is responsible for building output files from the source code in your project. Depending on the type of project, this output will be an executable and EEPROM image file, or a library.

For more detailed information about general build system concepts in the IDE, see the *Integrated Development Environment User's Guide*.

2.2 Build Stages

The Ezairo 7100 system consists of four processors, each of which is programmable to some degree. Build tools for the CFX, HEAR, Filter Engine, and ARM Cortex-M3 cores are integrated into the managed build system, under the SK4 ELF Toolchain. The automatic build process is divided into three stages:

- Generating any assembly source files from Filter Engine source and/or ARM Cortex-M3 processor subprojects
- Compiling or assembling the source files into object files
- Combining the object files to produce the final project output

The build system uses the Toolchain Driver and object librarian to create the output files by performing the following operations:

- Assembler source and header files are generated if necessary from the Filter Engine assembler, HEAR assembler, or possibly the ELF2CFS tool if this project includes ARM Cortex-M3 processor source code.
- The build system uses the assembler stage of the Toolchain Driver to assemble source files into object files.
- For executable projects, the build system uses the linker stage of the Toolchain Driver to combine the object files into an executable. The object files created by the HEAR assembler and the Filter Engine assembler are also linked at this point.
- For library projects, the build system uses a librarian tool to combine the object files into a library. The object files created by the HEAR assembler and Filter Engine assembler are also combined at this point.

To construct EEPROM images, the build system uses the NVMM in the IDE to create loadable non-volatile images for the Ezairo 7100 EEPROM. The NVMM is described in Chapter 8, "Non-Volatile Memory Management" on page 57. Tools for working with EEPROM images are described in Chapter 9, "Non-Volatile Image Tools" on page 76.

2.3 Build Settings

You can access your project's build settings by performing the following operations:

1. In the Project Explorer view, right-click on your project and select **Properties**.
2. Expand **C/C++ Build**.
3. Select **Settings**.

This editor page (shown in Figure 1) allows you to configure tool-specific build settings as well as create and edit configurations for your project build. Configurations contain instances of build settings. You can have as many configurations as you like. To create, rename, or delete configurations click on the **Manage Configurations...** button beside the configuration selection drop-down.

Build settings control the behavior of the various stages of the project build. In addition to general categories and settings, each build tool has a set of categories containing build settings specific to that tool. To edit your build settings, click on the category on the left and edit the settings on the right.

Using the **Restore Defaults** button you can reset all settings in the current build configuration back to their default value.

The Settings Editor allows you to freely edit the build settings in a working copy of the project's build settings. When you are finished editing, save all the information in the editor by clicking **Apply**.

For more information about the build settings for a processor other than the CFX, or to build a project from a command line, see one of the following manuals:

- *HEAR Configurable Accelerator Reference Manual*
- *ARM Cortex-M3 Processor Toolchain Reference*
- *Filter Engine Reference Manual*

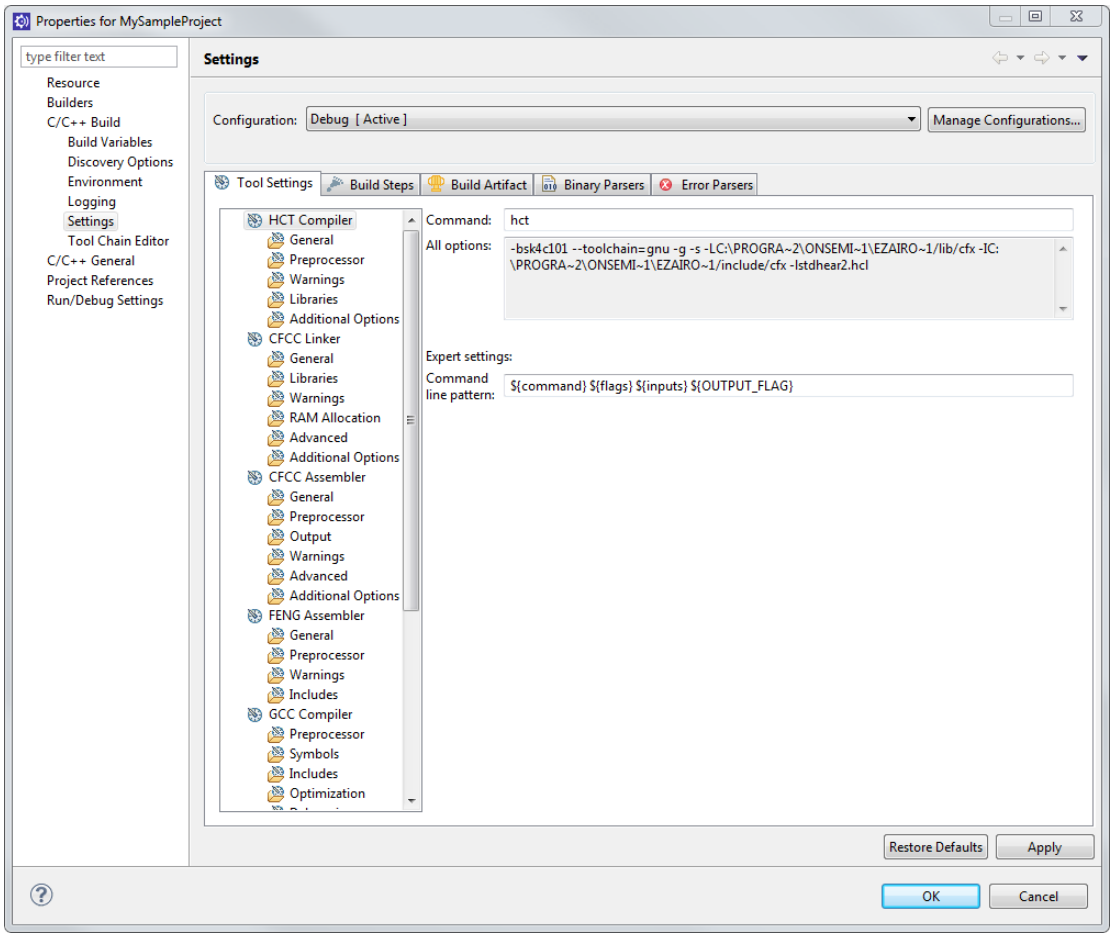


Figure 1. Build Settings Dialog

2.4 Build Variables

Build variables are variables that are filled in by the IDE before your project is built. They are of the format ${**VariableName**}. They can be used in your project's build settings to easily maintain common settings across various tools. There are several build variables supplied by default that are used in the build system.

To access build variables:

1. Right-click on your project and select **Properties**
2. Expand **C/C++ Build**
3. Select **Build Variables**
4. At this point you can add new variables. However, if you want to change supplied default variables, select the **Show system variables** check box.

Some of the supplied build variables that you will find of interest are:

- chip.id
- edk.include

- edk.lib
- target.architecture

2.5 AUTO-BUILD

The IDE Workbench can automatically rebuild a project when one of the following occurs:

- The project is created
- A file included in the build (or one of its dependencies) is changed or deleted
- A valid input file is created or added to the project
- The project is cleaned by selecting **Project** > **Clean...**
- A dependent project is built

As part of this project rebuild, the build system performs dependency calculations to determine what needs to be rebuilt, and what is already up-to-date. This allows the build system to rebuild only those files that have changed since the last build. You can continue to edit your source code while the build is running, and any build errors are displayed in the editors and the Project Explorer, Console and Problems views.

Sometimes, you might want to turn off this auto-build feature, especially when making a large number of changes where you know that the source code will not build correctly until you are done. Select **Project** > **Build Automatically** to enable or disable the auto-build feature.

Furthermore, you have to set auto-building for each project. Do the following:

1. Right-click on the project name in the Project Explorer view, and choose **Properties**.
2. Select **C/C++ Build** from the list on the left.
3. Click the **Behaviour** tab.
4. Select **Build on resource save (Auto build)** and click **OK**.

If **Project > Build Automatically** is already enabled, the project will immediately build.

CHAPTER 3
Running the Command-Line Tools

3.1 Introduction

This chapter provides a description of CFCC and its global options. Detailed information regarding specific stages of compilation can be found in later chapters in this manual.

CFCC is used by the builder in the IDE (see Chapter 2, “Building Software with the IDE” on page 10 for details). The default build settings should suffice in most cases, but if you need to modify your project defaults or use options that are not available through the builder preferences, read this chapter.

3.2 The Stages of Compilation

CFCC performs three stages of processing when transforming source code into executable machine code:

- Preprocessing
- Assembling
- Linking
- Archiving (optional)

CFCC accomplishes this processing by utilizing several lower level tools, and for this reason is often referred to as a *Toolchain Driver*.

3.2.1 Preprocessing

CFCC preprocesses source files using a standard C preprocessor.[2] Several extensions to the set of directives are also available. Preprocessing is discussed further in Chapter 4, “Preprocessing Source Code” on page 20.

3.2.2 Assembling

The assembler converts preprocessed source code (consisting of instructions, labels, and assembly directives) into binary form, stored in relocatable object files suitable for linking. The assembly language and build options are discussed in Chapter 5, “Assembling Programs” on page 30.

3.2.3 Linking

The relocatable object code produced in the assembling stage is converted into executable absolute code in the linking stage and is written to an executable output file. Collections of previously assembled relocatable object files (called *libraries*) can be created with CFX-AR and then linked in this linking step. Linking is discussed further in Chapter 7, “Linking Object Modules” on page 51.

3.2.4 Archiving/Librarian

CFCC drives CFX-AR (a GNU-based librarian) to produce library files. It accepts object files as input as well as metadata files associated with those objects.

2. ISO 9899-1990 Standard for the C language (also known as C90).

3.3 CFCC USAGE

Which of the three stages of compilation (preprocessing, assembling, and linking) CFCC performs on an input file depends on its filename extension. Table 1 identifies accepted file extensions and the stages of processing performed.

Table 1. Actions on File Extensions

File Extension	Description
.cfs	Assembly language source file. CFCC performs preprocessing, assembling, and linking.
.cft	Preprocessed assembly language source file. CFCC performs assembling and linking.
.o, .a	Relocatable object code files and libraries. CFCC performs linking.
.orc	Metadata files for archiving produced with the same name as the *.o* file. The CFCC linker looks for it by its object name; you never have to specify it as input to the CFCC linker.

Linking is always the last stage unless you use one of the `-c` or `-E` options to avoid it (or unless compilation errors stop the process). For the link stage, all *.o* files corresponding to source files and `-l` libraries are passed to the linker. The output of linking has a default extension of *.cfe*.

3.3.1 Syntax

CFCC accepts the following syntax:

```
cfcc options-and-files
```

Where **`options-and-files`** indicates one or more of the CFCC preprocessing, assembling, or linking options, including input filenames. You can find CFCC options in Table 2 on page 16. Options for the other stages appear in the chapters of this manual devoted to those stages.

Some options can only be bound to one value, such as `-b` to specify a target machine, and `-o` to specify an output file name. In the event that one of these options is specified several times, CFCC always uses the last option specified.

Some examples of CFCC usage:

- To assemble and link the executable *foo.cfe* from *foo.cfs*:

```
cfcc -bSK4 --toolchain=gnu -g --strictnop foo.cfs
```

- To assemble *foo.cfs* to *foo.o*:

```
cfcc -c -bSK4 --toolchain=gnu -g --strictnop foo.cfs
```

- To preprocess *foo.cfs* (prints to standard output):

```
cfcc -bSK4 --toolchain=gnu -E foo.cfs
```

- To build *app.cfe* from *foo.cfs* and *bar.cfs*:

```
cfcc -bSK4 --toolchain=gnu -g --strictnop -o app.cfe foo.cfs bar.cfs
```

- To generate a listing file called *foo.lst*:

```
cfcc -c -bSK4 --toolchain=gnu -g --strictnop --lst foo.cfs
```

- To generate a library file called *lib.a*

```
cfcc -a r --toolchain=gnu -v -o lib.a libobject1.o libobject1.orc
```

The `--strictnop` and `-g` options, for strict CFX instruction matching and debugging information respectively, are recommended for all assembly commands.

3.3.2 CFCC - Toolchain Driver Options

Table 2 describes some of the options you can use to specify the general behavior of CFCC. The following chapters contain specific options for each stage of compilation.

Table 2. CFCC Options

CFCC Option	Description
filename	The file or files to be preprocessed, assembled, and linked. The file extension determines which actions are to be performed (see Table 1 on page 15). At least one file name must be specified for CFCC to process.
`-c`	Assemble the source files, but do not link. The compiler outputs an object file for each source file. By default, CFCC makes the object file name for a source file by replacing suffixes such as `.cft`, `.cfs` with `.o`. You can override this default using the `-o` option. This option also causes CFCC to ignore any unrecognized input files (i.e., files that do not require assembly).
`-E`	Stop after the preprocessing stage; do not run the assembler. The output is preprocessed source code, which is sent to standard output by default. CFCC ignores input files that do not require preprocessing.
`-o` **file**	Place output in **file**. This action occurs regardless of the type of output CFCC is producing, whether executable file, object file, or preprocessed source code. Since only one output file can be specified, it does not make sense to use `-o` when compiling more than one input file, unless you are producing an executable as output. If you do not specify `-o`, an executable file called *a.cfe* is created by default, except when compiling a single file, in which case the executable is called **source**.*cfe* where **source** is the file name. The object file for **source**.*cfs* is put in *source.o*, and all preprocessed source is sent to standard output.
`-v`	Verbose mode. Prints (on standard error output) the commands executed to run the stages of compilation and runs all stages in verbose mode.
`-w`	Do not generate any warning messages.
`-g`	Produce debugging information.
`-b` **machine**	The argument **machine** specifies the target machine for compilation. Valid values for **machine** are `SK3` (default) and `SK4`. CFCC defines the preprocessor symbol `_TARGET_`**machine** to indicate its target.
`-h`	Print a usage message.
`-n`	Dry run. Performs no processing on input files. This is most useful in conjunction with the `-v` option.
`-x` **file**	Read options from **file**.
-a r**opts**	Archive the specified files. Valid input files are *.o* (object files) and *.orc* (object metadata). Specify `s` for **opts** to add an index to the archive as described in the GNU ar documentation. The tool outputs an ELF library. Available only with `--toolchain=gnu`

Table 2. CFCC Options (Continued)

CFCC Option	Description
`--ansi`	Force strict adherence to the C standard. This option causes the preprocessor to suppress support for `#macro`, `#endm`, `#rept`, `#endr`, `#set`, `#stripc`, `#stripasm`, `#nostrip`, `#nostripc`, and `#warning`. It also turns off support for C++-style comments.
`--toolchain=`**`<toolchain>`**	Selects the back end toolchain. Valid values for **`<toolchain>`** are `archelon` (default) and `gnu`.
`--version`	Outputs the name and version number of CFCC.

3.4 Error Codes

If CFCC runs successfully, it returns an error code of 0. If it fails because of an invalid option, it returns 2. If compilation fails due to input errors, it returns the non-zero code returned by the underlying tool. If it fails for any other reason, it returns an error code of 1.

3.5 Error Messages

This section lists and describes error messages you may encounter using the Toolchain Driver.

NOTE: This catalog of error messages is not comprehensive. It lists the error messages of CFCC proper, but not the messages generated by its underlying tools.

3.5.1 "Option error(option): message"

Printed when there is a problem reading the options from the command line. The **`option`** is the option in question. The **`message`** is a message describing the problem. This message is always followed by a usage message.

Example:

```
>cfcc --junk foo.cfs
Option error(junk): option --junk not recognized
Usage: cfcc [options] file...
Options:
  -h                              Display this information.
  (Use '-v -h' to display command line options of sub-processes)
  -c                              Assemble, but do not link
  -a r[opts]                      Archiver (GNU toolchain only), r flag must be specified
  -E                              Preprocess only; do not assemble or link
  -o <file>                       Place the output into <file>
  -v                              Verbose output
  -w                              Suppress warning messages
  -g                              Produce debugging information
  -b <machine>                    Target architecture: SK3 (default) or SK4
  --toolchain <toolchain>         Choose a backend toolchain (Archelon or GNU)
  -n                              Dry run, do not execute (most useful with -v)
  -x <file>                       Read options from <file>
  --ansi                          Force strict adherence to the C standard
  --version                       Display version information
```

3.5.2 "IO Error: message"

Printed when there is a problem reading options from an indirect file.

Example:

```
>cfcc -x foo.ind foo.cfs
IO Error: [Errno 2] No such file or directory: 'foo.ind'
```

3.5.3 “No input files”

Printed when no files are specified.

Example:

```
>cfcc
No input files
```

3.5.4 “Unknown target specified in -b: target”

Printed when an invalid target architecture is specified.

Example:

```
>cfcc -b FROB5000 foo.cfs
Unknown target specified in -b: FROB5000
Only SK3 and SK4 are valid.
```

3.5.5 “Cannot specify -o with -c and multiple compilations”

Printed when `-o` is specified with `-c` and multiple source files, as `-c` will generate an object file for each source file and `-o` specifies the name for a single output file. A similar error occurs when `-o` is specified with `-E` and multiple source files, as each source is preprocessed separately.

Example:

```
>cfcc -c --toolchain=gnu -o foo.o foo.cfs bar.cfs bat.cfs
Cannot specify -o with -c and multiple compilations
```

3.5.6 “Error: argument is not a file”

Printed when a specified argument is not a file.

Example:

```
>cfcc non-existent-file.cfs
Error: non-existent-file.cfs is not a file
```

3.5.7 “Don’t know how to build file: filename”

Printed when an argument has an unrecognized extension.

Example:

```
>cfcc foo.xxx
Don't know how to build file: foo.xxx
```

3.5.8 “System exception when running tool”

Printed if an underlying tool signals a system error.

3.5.9 "Error running tool: message"

Printed if there is a problem running an underlying tool.

Example (generated by renaming *cfpp.exe* to *pp.exe*):

```
>cfcc foo.cfs
Error running cfpp.exe: No such file or directory
```

3.5.10 "Unexpected error running tool"

Printed when some other error occurred when running an underlying tool. Users should never see this message.

3.5.11 "Error moving filename to destination: message"

Printed when there is an OS-level problem moving a generated file to a new location.

3.5.12 "Error copying filename to destination: message"

Printed when there is an OS-level problem copying a source file to the temporary working directory.

3.5.13 "An unexpected error has occurred in tool. Please contact ON Semiconductor customer support"

Printed when some unforeseen error occurs and one of the underlying tools crashes.

CHAPTER 4
Preprocessing Source Code

4.1 Introduction

Assembly language programs are often made more readable and maintainable through the use of macro facilities. In the CFX assembly language, these are provided through the use of a standard C preprocessor that is run prior to the assembler itself. In addition to the usual preprocessing constructs, the C preprocessor from ON Semiconductor implements extensions that are especially useful for assembly language programming.

The preprocessor filters the assembly language program provided as input, executing preprocessing directive lines and performing macro replacement on other input lines as they are passed through. In this way, the preprocessor is used to access hardware and system facilities provided by ON Semiconductor with the EDK, to define application-specific constants, to abbreviate repetitive or complicated constructs, and to specify conditional compilation.

This chapter describes how to run the preprocessor using CFCC, and provides detailed documentation of the preprocessing language implemented by the ON Semiconductor preprocessor. Particular focus is placed on the facilities that are extensions to the C standard, and details that are implementation-specific such as the location of `#include` files.

4.2 Running the Preprocessor

This section describes the CFCC preprocessing options. For information on CFCC syntax, see Table 2 on page 16.

4.2.1 Preprocessing Options

CFCC runs the preprocessor on input files that have a *.cfs* extension. Your use of CFCC options (see Table 2 on page 16) affects the behavior of the preprocessor, specifically those listed in Table 3.

Table 3. CFCC Options Affecting Preprocessor Behavior

CFCC Option	Effect on the Preprocessor
`-E`	Stop after the preprocessing stage; do not run the assembler. The output is preprocessed source code, which is sent to standard output by default. CFCC ignores input files that do not require preprocessing.
`-o` **`file`**	Place output in **`file`**. This action occurs regardless of the type of output CFCC is producing, whether executable file, object file, or preprocessed source code. Since only one output file can be specified, it does not make sense to use `-o` when compiling more than one input file, unless you are producing an executable as output. If you do not specify `-o`, an executable file called *a.cfe* is created by default, except when compiling a single file, in which case the executable is called **`source`***.cfe* where **`source`** is the file name. The object file for **`source`***.cfs* is put in *source.o*, and all preprocessed source is sent to standard output.

Table 3. CFCC Options Affecting Preprocessor Behavior (Continued)

CFCC Option	Effect on the Preprocessor
`-b` **`machine`**	The argument **`machine`** specifies the target machine for compilation. Valid values for **`machine`** are `SK3` (default) and `SK4`. CFCC defines the preprocessor symbol `_TARGET_`**`machine`** to indicate its target.
`--ansi`	Force strict adherence to the C standard. This option causes the preprocessor to suppress support for `#macro`, `#endm`, `#rept`, `#endr`, `#set`, `#stripc`, `#stripasm`, `#nostrip`, `#nostripc`, and `#warning`. It also turns off support for C++-style comments.

Options to CFCC that are specific to the preprocessing stage are described in Table 4.

Table 4. Preprocessing Options

CFCC Option	Description
`-C`	Preserves C-style (and C++-style) comments in the output.
`-t`	Refrains from outputting control lines which include the source line and file name.
`-S`**`name`**	Defines the identifier **`name`** as if you had included `#set name 1` at the beginning of the input file.
`-D`**`name`**	Defines the identifier **`name`** as if you had included `#define name 1` at the beginning of the input file.
`-D`**`name=str`**	Defines the identifier **`name`** as if you had included `#define name str` at the beginning of the input file.
`-U`**`name`**	Undefines the identifier **`name`** as if you had included `#undef name` at the beginning of the input file.
`-I`**`dir`**	Searches directory **`dir`** for included files if necessary.
`--ts`	Uses blanks to force explicit preprocessor token separation on output. This behavior allows other tools to implement standard C tokenization, but it is not the default since it is incompatible with the assembly language syntax.
`-Wundef`	Warns about undefined symbols used in `#if`, `#elif`, `#set`, and `#rept` directives. By default, the preprocessor substitutes the value 0 for symbols that are not defined when it is evaluating the expressions for these directives.

4.2.2 Predefined Macros

CFCC provides three predefined macros, in addition to those that are provided by any standard C preprocessor. They are:

__cf__

This macro is predefined to the decimal value 1. It is built into the preprocessor, so it should not be undefined or redefined.

TARGET<machine>

As described in Table 3 on page 20, the use of CFCC's `-b` option determines the specific name of the macro that is predefined. This macro is defined by CFCC using a built-in `-D` option to the preprocessor, so it can be undefined using the `-U` option, if desired.

_CFXASM_VER

This macro is predefined to a value which is a numeric representation of the version of the CFX assembler. The value is an integer equal to *major-version* * 100 + *minor-version* * 10 + *revision*, so for version 3.0.0 of the CFX assembler the value of `_CFXASM_VER` is 300.

This macro is defined by CFCC using a built-in `-D` option to the preprocessor, so it can be undefined using the `-U` option, if desired.

4.3 ON Semiconductor Preprocessing Language

CFCC's preprocessor provides the standard C preprocessing facilities, plus extensions that enhance its effectiveness as the macro language for CFX assembly language programming.

The following sections describe the enhanced C preprocessing language from ON Semiconductor, with emphasis on detailed descriptions of the facilities that are extensions to the C standard.

4.3.1 Preprocessing Directive Summary

Preprocessor actions are initiated with directives that begin with the hash character '#' as the first non-blank character in a line. (Note that comments are considered blanks.) Table 5 summarizes the directives that are supported by the preprocessor. The directives at the end of the table are extensions to standard C.

Table 5. Preprocessor Directives

Directive	Description	Extension to C
`#include` **`header-file`**	Process the header file as part of the input.	no
`#define` **`name text`**	Define a one-line macro without arguments.	no
`#define` **`name(args) text`**	Define a one-line macro with arguments.	no
`#undef` **`name`**	Remove the definition of a macro.	no
`#if` **`expression`**	Process following code if **`expression`** is non-zero.	no
`#ifdef` **`name`**	Process following code if **`name`** is defined.	no
`#ifndef` **`name`**	Process following code if **`name`** is not defined.	no
`#elif` **`expression`**	Process `else if` code if previous condition was zero and **`expression`** is non-zero.	no
`#else`	Process `else` code if previous condition was zero.	no
`#endif`	End a conditional compilation section.	no
`#error` **`text`**	Display diagnostic message with **`text`**, and exit.	no
`#line` **`number "file"`**	Set preprocessor's current line number and file, into `__LINE__` and `__FILE__`.	no
`#pragma` **`text`**	Do nothing. The preprocessor implements no `#pragma` directives, so **`text`** is ignored.	no
`#`	Do nothing. The null directive is ignored.	no
`#macro` **`name`**	Define a multi-line macro without arguments.	yes
`#macro` **`name(args)`**	Define a multi-line macro with arguments.	yes
`#endm`	End a multi-line macro.	yes
`#set` **`name expression`**	Define a numeric macro.	yes

Table 5. Preprocessor Directives (Continued)

Directive	Description	Extension to C
`#rept` **`expression`**	Start a block that will repeat **`expression`** times.	yes
`#endr`	End a repeated block.	yes
`#warning` **`text`**	Display diagnostic message with **`text`**.	yes
`#stripasm`	Remove assembly-style comments (beginning with semicolon (`;`)). Do not use with the new CFX assembler.	yes
`#nostrip`	Do not remove assembler comments (default).	yes
`#stripc`	Remove C and C++ comments (default).	yes
`#nostripc`	Do not remove C and C++ comments.	yes

4.3.2 Preprocessing Procedure

This section provides a brief, high-level overview of the procedure used by a C preprocessor (including CFCC's extended C preprocessor) for processing an input file.

Preprocessing is divided into the following conceptual phases:

Line-folding

Each instance of a backslash character \, without any other characters following it on the source file line, is deleted and the following line is joined to the line containing the backslash character to form a logical source line.

Tokenizing and comment removal

The logical lines of the source file are decomposed into tokens and white-space characters (which includes spaces, tabs, form feed, and comments; the latter are replaced with a single space character). The tokens are standalone lexical units such as names, punctuation characters, operators, numbers, and strings.

Directive execution and macro replacement

Logical lines are processed in order. Each line that is a preprocessing directive is executed.

A `#include` directive causes the specified header file to be preprocessed from phases 1 to 3, recursively.

`#if`, `#ifdef`, and `#ifndef` directives introduce a conditional compilation block, wherein at most one clause is true and therefore fully executed/scanned as described here; the other clauses, until the corresponding `#endif` directive, are scanned only to find corresponding `#elif`, `#else`, or `#endif` directives, or nested conditional compilation blocks.

Non-directive lines are scanned for macros that have been defined, and these are replaced with the text corresponding to the macro's value. After this replacement, the new text is re-scanned for further macros.

These preprocessing phases occur in order. Each preprocessor directive must occur on a single logical line, but line-folding allows this logical line to correspond to multiple physical lines (if the definition of the macro is long, for

example). Similarly, since comment removal precedes directive execution, you can intersperse comments within directives, if necessary.

4.3.3 Lexical Extensions

The preprocessor from ON Semiconductor provides some extensions to the C standard at the lexical level. These improve the flexibility of the preprocessor for use as the CFX assembly language macro processor.

4.3.3.1 Dollar in Names

The dollar (`$`) character is allowed in any position (including the first) of a name token. Thus, you can use `$` in the same way as the letters and _ in macro names. This usage is compatible with the assembler, which also treats `$` as an alphabetic character in names. See Section 5.3.1, “Lexical Conventions” on page 34.

4.3.3.2 Comment Removal

By default, the preprocessor removes C-style and C++-style comments (delimited with `/*` and `*/`, and `//` and newline, respectively) when translating the input to the output. If CFCC’s `--ansi` option is used, C++-style comments are not removed or recognized, and the `#stripc` and `#nostripc` directives (see below) affect the handling of C-style comments only.

If the `#nostripc` directive is used, the preprocessor does not remove C-style and C++-style comments when translating the input to the output. These comments are still recognized—their contents do not have macro replacement performed on them and they are not treated as tokens (which could cause syntax errors for comments in directives)—but they are copied through from the input to the output. In some cases they are not treated as space characters (as preprocessing phase 2 requires; see Section 4.3.2, “Preprocessing Procedure” on page 23), so the behavior of the preprocessor can change when comments are interspersed with directives. It is recommended that comments not be interspersed with directives when `#nostripc` is in effect.

The `#stripc` directive causes the preprocessor to remove C-style and C++-style comments, restoring the default behavior.

The `-C` preprocessor option is equivalent to using the `#nostripc` directive at the start of the input file.

4.3.3.3 Escaping the Hash Character

In some uses of the preprocessor, it is desirable for the output file to contain hash (`#`) characters. For example, you might use the preprocessor to automatically generate a source code file from a template, with the output source code file containing preprocessor directives.

This usage is supported by the preprocessor. Hash characters can be “escaped” by preceding them with the backslash (`\`) character. When preceded in this way, the hash character is not interpreted as the directive lead-in character, so it is passed through from the input to the output.

4.3.4 Preprocessor Expressions

The preprocessor evaluates numeric expressions that are the operands of the `#if`, `#elif`, `#set`, and `#rept` directives. These expressions involve integer constants (which may be generated by macro replacement), operators, and parentheses for grouping. The operators available are the usual C expression operators, as shown in Table 6.

Table 6. Operators, By Group

Operator	Description
`defined name` `defined(name)`	Unary "is-macro" predicate
`+ - ~ !`	Unary operators
`* / %`	Multiplicative operators
`+ -`	Additive operators (binary)
`<< >>`	Shift operators
`< > <= >=`	Relational operators
`== !=`	Equality operators
`&`	Bitwise AND operator
`^`	Bitwise exclusive OR operator
`|`	Bitwise inclusive OR operator
`&&`	Logical AND operator
`||`	Logical OR operator
`? :`	Conditional operator (ternary)

Expressions are evaluated using C precedence rules, which are indicated by the rows in Table 6. Operators with higher precedence are at the top of the table, and operators with equal precedence appear in the same row.

The `defined` predicate returns 1 if the name has been defined as a macro using `#define`, `#macro`, or `#set`, without an intervening `#undef`. Otherwise, it returns 0.

Arithmetic operations use 64 bits; signed numbers are represented using two's complement. The right-shift operator (`>>`) is "arithmetic" when applied to a signed, negative value. That is, high-order bit positions are filled with copies of the sign bit. The division operator (`/`) and remainder operator (`%`) perform integer division that truncates towards zero, for negative and positive operands.

4.3.5 Directive Extensions

The following sections describe how this particular preprocessor implements the directives and any non-standard directives.

4.3.5.1 Details of #include File Searching

There are two forms of the #include directive:

```
#include <filename>
#include "filename"
```

These forms may also be generated through macro replacement:

```
#if WIDGET_PRODUCT
    #define PRODUCT_INCLUDE "widget.cfi"
#else
    #define PRODUCT_INCLUDE <standard_product.cfi>
#endif
```

```
#include PRODUCT_INCLUDE
```

The preprocessor uses the following method to search for **`filename`**:

- If **`filename`** specifies an absolute path, then the preprocessor includes the specified file, and the search is done. An absolute path is one that begins with a drive specification such as `C:`, a backslash (`\`), or a forward slash (`/`).
- Otherwise, **`filename`** specifies no path or a relative path. If **`filename`** is surrounded by double quotes, the preprocessor appends **`filename`** to the directory path of the file that contains the `#include` directive, and attempts to open the resulting file. If the file can be opened, then it is included, and the search is complete.
- Otherwise, **`filename`** cannot be found relative to the directory of the file containing the `#include`, or **`filename`** is surrounded by angle brackets. The preprocessor then appends **`filename`** to each specified include directory, in order, and attempts to open the resulting file. If the file can be opened, then it is included and the search is complete; otherwise the preprocessor tries the next directory in the list of include directories.

  The list of include directories is formed from the `-I` options on the command line (in order from left to right).

4.3.5.2 Comment Removal

The preprocessor provides the `#stripasm`, `#nostrip`, `#stripc`, and `#nostripc` directives to control the removal of semicolon comments and C/C++-style comments. Full details regarding these four directives may be found in Section 4.3.3, “Lexical Extensions” on page 24.

4.3.5.3 Generating a Warning with #warning

The `#warning` directive produces a similar diagnostic message to that produced by the `#error` directive. However, the preprocessor continues processing the input and generating output; it does not terminate due to the `#warning` directive, as it does with the `#error` directive.

4.3.5.4 Repeating Text Blocks Using #rept

The `#rept` and `#endr` directives enclose a block of text lines that are repeated in the output. The number of repetitions is given by the **`expression`** operand to `#rept`.

The repeat count expression is evaluated as described in Section 4.3.4, “Preprocessor Expressions” on page 24. The result is truncated to 16 bits (the high 48 bits are simply ignored) and treated as a signed 16-bit number. This number is the repeat count; it must be greater than or equal to zero.

The lines between `#rept` and `#endr` are only processed through phases 1 and 2 of the preprocessing procedure (see Section 4.3.2, “Preprocessing Procedure” on page 23) as they are collected. Preprocessing phase 3 is then performed on the lines each time they are repeated in the output. This means that execution of preprocessing directives and macro replacement are performed on each line, each time it is output.

The `#set` directive is particularly useful within a repeated section, as a loop index for example.

4.3.5.5 Defining Multi-Line Macros with #macro

The `#macro` and `#endm` directives are used to define multi-line macros.

The `#macro` directive follows the same rules as the `#define` directive for specifying arguments; that is, you can define multi-line macro arguments with one of the following uses of parentheses:

- No arguments (and no parentheses for an argument list)
- Empty parentheses (which defines a function-like macro with no arguments)
- One or more argument names in parentheses

Unlike `#define`, however, the replacement text for a multi-line macro does not appear on the same line as the directive. In other words:

```
#macro
 //Replacement text starts at the beginning of this line
...
 //Replacement text ends immediately before this newline
#endm
```

The lines between `#macro` and `#endm` are only processed through phases 1 and 2 of the preprocessing procedure (see Section 4.3.2, “Preprocessing Procedure” on page 23) as they are collected. Preprocessing phase 3 is performed on the lines each time the macro is invoked. In other words, an invocation of the macro is replaced by its replacement text (also known as the macro’s “body”). This means that execution of preprocessing directives and macro replacement are performed on each line of the macro, each time it is invoked.

Like single-line macros defined with `#define`, multi-line macros may not be redefined unless the new definition is preceded by `#undef` to remove the existing definition.

4.3.5.6 Defining Numeric Macros with #set

The `#set` directive defines a numeric macro. The **`expression`** operand to `#set` must be a correctly formed expression, since `#set` evaluates the **`expression`** as described in Section 4.3.4, “Preprocessor Expressions” on page 24 and stores the result as the replacement value for the numeric macro.

When a numeric macro is invoked, the replacement text for the macro is the replacement value set in the most recent `#set` directive for the macro, printed in signed decimal.

You can redefine numeric macros with a new expression, or redefine them as single-line or multi-line macros, without any warning occurring.

4.3.5.7 Comparison of Macro Types

The ON Semiconductor preprocessing language provides three different methods with which to define a symbolic name for some sort of value:

- The standard `#define` directive
- The extension directive `#macro`
- The extension directive `#set`

This section compares these three macro-defining directives, presenting the pros, cons, and intended usages of each.

The `#macro` directive is quite similar to `#define`, in that it defines a macro with a textual replacement, optionally including arguments that are supplied at the time the macro is invoked and replaced.

The obvious difference between `#macro` and `#define` is that `#macro` allows multiple lines in the replacement text. Since the CFX assembly language is fairly line-oriented (unlike most high-level programming languages like C, in which spacing and line distinctions are matters of style, not necessity), and preprocessor directives themselves are

line-oriented, multi-line macros are necessary for typical assembly language macro tasks where a macro often contains many instructions.

The second key distinction between `#macro` and `#define` is that preprocessor directives can be embedded in a multi-line macro defined with `#macro`, and these directives are executed at the time that the macro is invoked and replaced. You can use the conditional compilation directives to test for features, or particular arguments or combinations of arguments (which may require special code sequences, or may be erroneous, for example). You can use the `#define` and `#set` directives to build up values that are used later in the macro body, such as in an immediate load instruction. They are also helpful for creating unique labels for each invocation of a macro. Here is an example of using `#set` to increment a counter that is used to generate unique label names:

```
#macro Sys_SPI_WaitDone

    // Generate unique loop label
    #ifndef LABEL_NAME
        #set COUNTER 0
    #endif
    #set COUNTER COUNTER+1
    #undef LABEL_NAME
    #define LABEL_NAME paste( _SPI_WaitDoneLoop, COUNTER )

    add sp, 1           >>> store x[sp - 1], x1   // Store X1 on stack
    LABEL_NAME:
        load x1, p[D_SPI_CTRL]
        bittst x1, SPI_CTRL_BUSY_POS ||| nop
    goto.0d LABEL_NAME, carry
    load x1, x[sp - 1]  >>> add sp, -1             // Restore X1 from stack

#endm
```

NOTE: Some of the names in the example above were modified to fit on the page. To see the full code, look at *sk4_sys_spi.cfi*.

Due to their multi-line nature, however, macros created with `#macro` are most often “procedure-like” macros, in that they expand to a number of instructions, and are typically invoked on their own in source code, as in:

```
SendByteOutUART(0)
```

Single-line macros created with `#define` can often be invoked in a “function-like” manner. For example:

```
#define POW2(n)  ( 1 << (n) )

load a0.h, POW2(12)
```

In fact, since a multi-line macro containing only a single line of text will contain no newline characters (see the precise definition of the body of a macro created with `#macro`, in Section 4.3.5.5, “Defining Multi-Line Macros with #macro” on page 26), it is actually possible to create the `POW2()` macro with either `#define` or `#macro`. However, if the body of a multi-line macro is more than one line, because it uses multiple calculation steps, or `#rept` blocks, or conditional compilation with `#if`, then it is not possible to invoke the macro in a function-like manner. You can work around this limitation by defining a procedure-like macro that generates a `#define` or `#set` directive with the desired result value; you can then use the macro created with the `#define` or `#set` directive as an operand in an instruction, for example.

Numeric macros created with `#set` are useful for computation. The macros created by `#define` and `#macro` are purely textual. They do not evaluate their replacement text even if it has the form of a numeric expression. Thus it is difficult or impossible to compute a value that has an iterative nature (so would naturally be “built up” using `#rept`, for example), or to create a value using several conditional steps where bitfields are ORed in, one at a time, conditionally.

The `#set` directive works well for iterative tasks, as the macro can be redefined at any time, typically in terms of itself:

```
#set index 0
#rept 42
    <do something>

    #set index index + 1
#endr
```

The limitation with numeric macros is that they can only be numeric. There is no way to repeatedly append to a macro that has a textual value, such as a string.

CHAPTER 5
Assembling Programs

5.1 INTRODUCTION

The CFX assembler converts CFX assembly language programs into relocatable object files, which can be linked together to form full executables to run on the CFX DSP. The details of the relocatable object file format are described in Section A.3, “Relocatable Object File Format” on page 84.

CFX assembly language is a typical “old-style”, mnemonic-based assembly language, with syntax to support the CFX DSP’s parallel instruction set. Prior to assembling, assembly language programs are usually preprocessed to provide macro facilities.

5.2 RUNNING THE ASSEMBLER

In most cases, CFCC needs to preprocess assembly language source files. Preprocessing is needed to expand macros within the files, and to obtain information required to properly assemble the files, including hardware and operating software definitions.

CFCC automatically preprocesses assembly language input files with a *.cfs* extension. When CFCC encounters files with a *.cft* extension, it treats them as assembly language source code that does not require preprocessing, and assembles them without preprocessing.

Your use of CFCC options (see Table 2 on page 16) affects the behavior of the assembler, specifically those listed in Table 7. Also, since most assembly language files are preprocessed prior to assembly, you will often use CFCC’s preprocessor options as well (see Table 4 on page 21).

Table 7. CFCC Options Affecting Assembler Behavior

CFCC Option	Effect on the Assembler
`-c`	Assembles the source files, but does not link them. The assembler outputs an object file for each source file. This option also causes CFCC to ignore any unrecognized input files (i.e., files that do not require assembly). If `--toolchain=gnu`, CFCC automatically generates a metadata file (*.orc*) with the same name as the object file for archiving purposes.
`-o` **`file`**	Places output in the file specified as a parameter. This action occurs regardless of the type of output CFCC is producing, whether executable file, object file, or preprocessed source code. Since only one output file can be specified, it does not make sense to use `-o` when compiling more than one input file, unless you are producing an executable as output. If you do not specify `-o`, the default is to put the object file for **`source`***.cfs* in **`source`***.o*, where **`source`** is the file name. In the case of the assembler, if an object file is specified as the output file, a metadata file is also produced with the same name but with a *.orc* extension.
`-g`	Produces debugging information.
`-b` **`machine`**	The argument **`machine`** specifies the target machine for compilation. Valid values for **`machine`** are `SK3` (default) and `SK4`. CFCC defines the preprocessor symbol `_TARGET_`**`machine`** to indicate its target.

5.2.1 Assembler Options

In addition to the common CFCC options listed in Table 7, CFCC accepts assembler-specific options. Available options are listed and described in Table 8.

Table 8. CFCC Assembler Command Line Options

CFCC Option	Description
`--lst`	Produces a program listing, showing the values generated for each instruction. Places this listing in the file `name.lst`.
`--strictnop`	Arithmetic-Move instructions must contain both the Arithmetic instruction part and the Move instruction part. If an instruction part is not used, NOP must be used to complete the Arithmetic-Move instruction. By default, the CFX assembler accepts Arithmetic and Move instructions as if they are full Arithmetic-Move instructions. The assembler provides the missing NOP.
`--strict`	Disables support for synthesized and synonym instructions. By default, the assembler accepts alternate forms for some machine instructions, as described in Section 5.3.2.2, “Synonyms and Synthesized Instructions” on page 36.
`--no-check`	Disables checks for restricted instruction combinations. By default, the assembler produces error messages and fails if it finds any incorrect or dangerous code combinations, as described in Section 5.2.2, “Instruction Restriction Checks” on page 31. We do not recommend using `--no-check` to disable *all* restriction checking. If some of the restriction checks are too stringent (i.e., the assembler produces error messages for instruction combinations that you know are valid), it is better to disable those specific restriction checks using the `.check` directive (see Section 5.3.3.5, “Directives for Controlling Checking” on page 43).
`--dotsegment`	Enables processing of the `.segment` directive by the CFX assembler. By default, the assembler does not accept the `.segment` directive because the `.pseg`, `.xseg`, `.yseg` and `.xyseg` directives are simpler to use, and they prevent errors with mismatched memory types and word sizes.

5.2.2 Instruction Restriction Checks

The CFX DSP’s parallel instruction set and “visible” pipeline are designed for high performance, with minimal hardware restrictions. Pipeline integrity is the responsibility of software, so CFX programs must avoid certain instruction combinations and concurrent operations to ensure that the DSP executes instructions with their documented behavior.

The CFX assembler, which CFCC uses to assemble programs, checks all of the pipeline- and parallelism-related instruction restrictions that are documented in the *CFX DSP Architecture Manual*. The assembler takes a cautious approach to restriction checking. It produces an error message for code that *might* be restricted, if it cannot verify that no restrictions are violated. The following sections briefly describe the checks that the assembler performs, and how to interpret or disable the error messages that are produced.

5.2.2.1 Restriction Categories

The *CFX DSP Architecture Manual* provides the precise details regarding instruction restrictions. The general categories, with some notes to help understand the restriction checks, are as follows:

Multiplication Restrictions

All of the CFX DSP’s multiplication instructions take two cycles to complete, with the second (ALU) cycle executing in parallel with the following instruction. The parallelism of that ALU

operation—which occurs for both *multiply-only* (`MUL`, `MULNEG` and related mnemonics) and *multiply-accumulate* (`MULADD`, `MULSUB` and related mnemonics) instructions—introduces several potential conflicts related to timing and to resources such as registers and ALUs.

The "following instruction" is often the instruction at the next address in P memory, but when the multiplication is in the last delay slot of a control-flow instruction, the following instruction is at the target address of the control-flow instruction. For multiplications in the last delay slot of a conditional branch or as the last instruction of a hardware loop, there are two potential following instructions.

Delay Slot Restrictions

Delay slots allow the DSP's pipeline to read ahead, to avoid stalling execution when program flow changes. Since program flow is involved, further program-flow instructions cannot be used in delay slots. To simplify the decoding that is done while reading ahead, the CFX also forbids the use of multi-cycle *Long* instructions in delay slots.

The instructions in the delay slots of the `RETURNI.2D` instruction have several other restrictions because `RETURNI.2D` is restoring the status register `SR` during its delay slots.

Hardware Loop Restrictions

The CFX DSP's zero-overhead hardware loops use the pipeline to "execute ahead" in a similar fashion to the read-ahead that the CFX does for program-flow delay slots. The program's flow changes at the end of the hardware loop, so there are similar restrictions at the end of hardware loops: control-flow instructions are forbidden. Furthermore, nested hardware loops cannot end on the same instruction.

Concurrent Register Updates

With the exception of the status register `SR`, multiple concurrent updates to a CFX register do not work. Such updates could occur within an instruction since the *Arithmetic* and *Move* parts of an *Arithmetic-Move* instruction execute in parallel, as do the *Dual* forms. Concurrent updates could also occur due to the two-cycle nature of multiplication instructions.

5.2.2.2 Restriction Error Messages

Restrictions are most often an interaction between two instructions:

- A *containing* instruction, such as a multiplication, a `LOOP`, or a control-flow instruction that has delay slots.
- An *erroneous* instruction; the instruction which—due to the restriction—cannot be used where it is, relative to the containing instruction.

When a restriction is violated, the assembler produces an error message that refers to the erroneous instruction. The message has the following format:

```
"<file>.cfs", line <L>/<C>: error: <message>
```

L is the line number of the erroneous instruction, and *C* is the character position within the line at which the problem was detected. *C* is often very useful for pinpointing which register or operand is at fault; even when *C* refers to a whole instruction, it can be useful for determining which sub-instruction of an *Arithmetic-Move*, *Move-Move* or *Dual* instruction is violating the named restriction.

The CFX assembler takes a fairly cautious approach to restriction checking, producing errors for dangerous constructs that might be valid in certain conditions—conditions that the programmer can verify but the assembler cannot. All violations (or potential violations) of restrictions cause the assembler to produce errors—not warnings—so CFCC fails if any restriction is violated.

To disable a restriction check, use the `.check` directive. If you use `.check` to turn off checking on a containing instruction, *all* checks related to that containing instruction and instructions it "contains" are disabled. For example:

```
    goto.2d handle_negative if ltz
    nop ||| load y0, y[yp0++]         // [delay slot 1]
    muladd a0, x0, y0 ||| nop         // [delay slot 2]

    // The following STORE is valid -- storing the value
    // of a0 before adding x0 * y0 -- because interrupts
    // are disabled in this interrupt-handler code.
    .check off
    store x[X_PREVIOUS_PRODUCT], a0
    .check on

    ...

handle_negative:
    load xp0, X_PREVIOUS_PRODUCT
```

The assembler does not know that interrupts are disabled because your code is part of an interrupt handler and consequently, that the `STORE` instruction is not a problem. (If an interrupt did occur after the `MULADD` was initiated, the `STORE` would store the new value of `a0`, *after* adding `x0 * y0`.) Therefore, without the `.check off` directive, the assembler would generate an error message.

If the `handle_negative` label was external to this file (declared with `.extern` or `.declare`), so the assembler could not check the multiplication restrictions at that label (it must do this because the `MULADD` is in the last delay slot of the `GOTO.2D`), a further message would be produced. This could be suppressed—probably along with a comment explaining why the code at the `GOTO.2D`'s target address does not violate multiplication restrictions—using `.check off` prior to the `MULADD` instruction. Turning off restriction checking on that single instruction would also suppress the error message for the `STORE`, because the `MULADD` is the containing instruction for the `STORE` (the `STORE` is one of the two instructions that executes during the second cycle of the `MULADD`).

NOTE: Using `.check off` to suppress restriction checks based on knowledge about the code is dangerous because no automated checks ensure that the code continues to work properly when it is changed. Use `.check off` sparingly, and monitor dependencies within the code carefully as the code evolves.

See Section 5.3.3.5, "Directives for Controlling Checking" on page 43 for more information about the `.check` directive.

5.3 CFX Assembly Language

The CFX assembly language is a typical mnemonic-based assembly language, with facilities for controlling memory segments, generating data tables, and referencing symbols defined externally, in addition to assembling CFX instructions. Details about the CFX instruction set may be found in the *CFX DSP Architecture Manual*.

5.3.1 Lexical Conventions

A name must begin with an alphabetic character, dot (.), underscore (_), or dollar sign ($). The rest of the name may consist of alphanumerics, underscore, or dollar sign, but no dots. The name "." is replaced by the current program counter value. A name can be up to 256 characters long, and all characters are significant for both internal and external linkage.

The assembler is case-sensitive for names defined in your program, such as labels and segments. You can give CFX instruction mnemonics, instruction options, register names and assembler directives in mixed case because they are case-insensitive. For example, you can spell the `XP1` register `XP1`, `xp1`, or `Xp1`.

Blanks and tabs are only needed to delimit tokens, where they are not already delimited by punctuation. For example, an instruction mnemonic is only recognized as such when you separate it from the first operand by at least one space or tab, since there is no punctuation between the mnemonic and the first operand.

When tokens are already delimited by punctuation, any number of spaces or tabs may appear between the tokens.

Continue very long lines by placing a backslash character at the end of the line, just before the newline character. CFCC implements this in the preprocessing stage; only use it in `.cfs` files, since these are preprocessed by CFCC. The CFX assembler allows lines to be split—without a backslash—when it is obvious that an instruction is unfinished: after operators in expressions and after punctuation (such as the commas between operands or the sub-instruction separators |||, >>> or !!). This is implemented in the assembler parser, so you can use it in both `.cfs` and `.cft` files. For example:

```
muladd a0, x0, y0 !!
        muladd b0, y1, x1  |||
                load x0, x[xp0+!] !!
                        load y1, y[yp0++]
```

Furthermore, the assembler allows you to write multiple instructions on a line, separated by semicolons. For example:

```
call.2d xp2
store y[Flag], 1;  add sp, 200  // [2 delay slots]
```

The preprocessor removes comments, and thus uses the C and C++ language conventions: between `/*` and `*/`, or from `//` to the end of the line.

Numbers are C-style and can be binary, hexadecimal, octal or decimal, as shown in Table 9.

Table 9. C-Style Numbers

Radix	Starting Characters	Permissible Characters
binary	`0b` or `0B`	01
hexadecimal	`0x` or `0X`	0123456789abcdefABCDEF
octal	`0`	01234567
decimal	non-zero digit	0123456789

The CFX assembler recognizes boolean constants: the keywords `FALSE` and `TRUE` (which are case-insensitive like other keywords) are numbers with the values 0 and 1, respectively.

5.3.2 Assembly Language Syntax

The CFX assembly language is line-oriented. After preprocessing and removal of comments, each line in an assembly language file consists of zero or more labels, followed by an optional instruction or directive. As mentioned in Section 5.3.1, “Lexical Conventions” on page 34, the CFX assembler automatically joins lines that are split after separators, and allows the semicolon to terminate or separate instructions on one line.

A label is a name, as defined in Section 5.3.1, “Lexical Conventions” on page 34, followed by a colon (`:`).

The syntax for directives is detailed in Section 5.3.3, “Assembler Directives” on page 39.

CFX instructions are written using the syntax shown in the *CFX DSP Architecture Manual*. Each instruction begins with a mnemonic, consisting of alphanumeric and dot (“.”) characters, which, depending on the instruction, might be followed by a comma-separated list of operands. The instruction mnemonic is separated from the operand list with spaces or tabs.

Typical instruction operands are labels, registers, conditions, constant expressions, and memory references.

5.3.2.1 Instruction Forms

CFX instructions come in three general forms, with the following syntax:

```
Long
Arithmetic  |||  Move
Move  >>>  Move
```

Both the **Arithmetic** and **Move** instruction classes have single and dual forms. **Dual Arithmetic** instructions are composed of an **X Arithmetic** instruction and a **Y Arithmetic** instruction:

```
Single Arith        |||  Move
X Arith !! Y Arith  |||  Move
```

Dual Move instructions are composed of an **X Move** instruction and a **Y Move** instruction:

```
Arithmetic   |||  Single Move
Arithmetic   |||  X Move !! Y Move
Single Move  >>>  Single Move
Single Move  >>>  X Move !! Y Move
```

All combinations of the different types are valid:

```
X Arith !! Y Arith  |||  X Move !! Y Move
X Move !! Y Move    >>>  Single Move
X Move !! Y Move    >>>  X Move !! Y Move
```

In the assembly language, each **Long** instruction and each instruction in the **Arithmetic-Move** and **Move-Move** instructions is written in the form of a typical assembly language instruction, with a mnemonic instruction name and operands (if any). The *CFX DSP Architecture Manual* contains a comprehensive list of all the instructions and their forms (**Long**, **Single Arithmetic**, **Y Move**, etc.)

The assembler recognizes **Arithmetic-Move** instructions that are missing either the **Arithmetic** or **Move** part (and the ||| connector), and assembles the **Arithmetic** or **Move** instruction along with a `NOP`, into an **Arithmetic-Move** instruction. The CFX assembler allows you to enable or disable these “optional NOPs” on a per-instruction basis using

the `.strictnop` directive. See Section 5.3.3.5, “Directives for Controlling Checking” on page 43 for more information.

If you use the `--strictnop` assembler option for a project, you cannot omit one part of an **Arithmetic-Move** instruction form in the CFX assembly source code for this project. In other words, you must fill in a NOP for an unused **Arithmetic** or **Move** part. For more information about this option, see Table 8 on page 31.

5.3.2.2 Synonyms and Synthesized Instructions

The CFX assemblers provide multiple forms for the same instructions or operands in cases where the alternate forms can improve program clarity. The following sections describe these *synonym* forms.

5.3.2.2.1 Synonym Instructions and Synthesized Instructions

Both CFX assemblers provide no-delay-slot forms for the control-flow instructions. These have no **`.nD`** suffix on their mnemonics (where **`n`** is 0, 1 or 2); they are provided to simplify the instruction set for new CFX developers and to speed initial development when optimization is not a concern.

These no-delay-slot instructions have the mnemonics `CALL`, `GOTO`, `LOOP`, `RETURN` and `RETURNI`. Where a zero-delay-slot form exists in the native instruction set (`GOTO.0D` and `RETURNI.0D`), the no-delay-slot form is a straightforward *synonym* for the machine’s **`.0D`** instruction. Where the native instruction set only has one-delay-slot or two-delay-slot instructions, the no-delay-slot forms are *synthesized* from the machine instruction followed by one or two `NOP` cycles.

The CFX assembler provides additional synonym instructions, as follows:

`NOT dst-reg, src-reg`
: The CFX does not provide an instruction for the unary logical complement operation. This *Single Arithmetic* synonym implements that operation by XOR’ing with -1 (which is all 1 bits in two’s complement form).

`MOVE dst-reg, src-reg`
: The CFX does not provide an unconditional instruction for copying between full accumulators (copying all 56 bits, as *Arithmetic* instructions do). This *Single Arithmetic* synonym implements that operation by XOR’ing with zero.

`SUB <immediate>`
: The CFX does not provide subtraction instructions with immediate operands. The assembler provides a `SUB` synonym for each `ADD` instruction that has an immediate operand. The synonym’s immediate operand is negated. Thus you can say:

```
sub sp, 5
```

instead of:

```
add sp, -5
```

The *CFX DSP Architecture Manual* provides more information about the syntax and implementation of these synonym and synthesized instruction forms.

The CFX assembler provides the `--strict` option (see Section 5.2.1, “Assembler Options” on page 31) and the `.strict` directive (see Section 5.3.3.5, “Directives for Controlling Checking” on page 43) for disabling and enabling the synonym and synthesized instructions.

5.3.2.2.2 Alternate Condition Forms

The CFX assembler provides synonyms for some of the conditions that you can use in conditional `GOTO` instructions and conditional *Arithmetic* instructions. For example, you can test whether the X Zero bit in the `SR` is set using any of the conditions `EQ`, `X_EQ`, `ZERO` or `X_ZERO`. The forms without the `X_` prefix are simpler and clearer if it is obvious from the context which ALU’s flags you are testing. Similarly, the `EQ` form is clearer if two values (registers, constants) have been subtracted or compared, while the `ZERO` form is more straightforward if a value has been compared to zero. For a complete list of synonyms that are available, see the *CFX DSP Architecture Manual*.

5.3.2.2.3 Addressing Mode Synonyms

The CFX assembler recognizes alternate forms for some of the core’s indirect addressing modes. Most of these synonym forms have two benefits: they use syntax that is the same as earlier DSP cores, and they are more concise (which can improve code readability when the forms are familiar). The indirect addressing modes are summarized in Table 10, where a memory-reference operand to X memory via `xp0` is used for illustration.

Table 10. Addressing Mode Synonyms

Base Addressing Mode Syntax	Synonym Addressing Modes
`X[ xp0 ]`	
`X[ xp0++ ]`	`X[ xp0+ ]` `X[ xp0 += 1 ]`
`X[ xp0-- ]`	`X[ xp0- ]` `X[ xp0 -= 1 ]`
`X[ xp0 += 2 ]`	
`X[ xp0 -= 2 ]`	
`X[ xp0 += xstep0 ]`	`X[ xp0+! ]`
`X[ xp0 -= xstep0 ]`	`X[ xp0-! ]`
`X[ xp0 +%= xstep0 ]`	`X[ xp0+%! ]`

5.3.2.3 Numeric Instruction Operand Syntax

Numeric operands occur in many types of instructions. Details regarding the size, signedness, and function of these numeric operands can be found in the *CFX DSP Architecture Manual*, while the following paragraphs describe the syntax and constraints that apply to all numeric operands.

Immediate operands have six different forms, which determine the range of values as listed below. Note that the last two operand forms are used by the subtract-immediate instruction synonyms (see Section 5.3.2.2, “Synonyms and Synthesized Instructions” on page 36 for more information about the subtract-immediate instruction synonyms).

- *immN* operands are put into *N*-bit fields with no defined signedness. They can therefore have values in the range -2^{N-1} to $2^{N}-1$. This is the union of the ranges of *N*-bit two’s complement signed numbers and *N*-bit unsigned numbers.
- *immNs* operands are put into signed *N*-bit fields, and can therefore have values in the range -2^{N-1} to $2^{N-1}-1$.
- *immNu* operands are put into unsigned *N*-bit fields, and can have values in the range 0 to $2^{N}-1$.

- *immNn* operands are put into negative *N*-bit fields. The encoded value is taken to be negative by the CFX; that is, it is extended upwards with 1 bit. These immediate operands can have values in the range -2^N up to -1.
- *immNsn* operands are negated and put into signed *N*-bit fields, and can therefore have values in the range $-2^{N-1}+1$ to 2^{N-1}.
- *immNp* operands are negated and put into negative *N*-bit fields. The encoded value is taken to be negative by the CFX; that is, it is extended upwards with 1 bit. These immediate operands can have values in the range 1 to 2^N.

In addition to allowing the range (as described above) that corresponds to the machine instruction's immediate field, the assembler uses the destination's size and signedness to determine the valid ranges. Most registers are taken as "signed or unsigned", so if the size of the immediate field matches the size of the register (as with the 24-bit `LOAD` immediate in the *Long* class), both signed and unsigned values are allowed. If the immediate field is smaller than the destination and is signed, the negative values in the range, sign extended up to the size of the destination, are allowed as unsigned values. Similarly, unsigned registers (such as pointer registers) and operands (for `CMPU.NX`, for example) can only be given unsigned values.

The assembler front end checks the value of an immediate operand if no labels are used in the expression. Out-of-range values trigger an error message. In some cases, such as bit indexes and shift counts, a tighter range is enforced as documented in the *CFX DSP Architecture Manual*.

Numeric operands are arithmetic expressions involving constants, symbols, operators, and parentheses for grouping. The operators available are the usual C expression operators, as shown in the Table 11.

Table 11. Operators, by group

Operator	Description
`+ - ~ !`	Unary operators
`* / %`	Multiplicative operators
`+ -`	Additive operators (binary)
`<< >>`	Shift operators
`< > <= >=`	Relational operators
`== !=`	Equality operators
`&`	Bitwise AND operator
`^`	Bitwise exclusive OR operator
`|`	Bitwise inclusive OR operator
`&&`	Logical AND operator
`||`	Logical OR operator

Expressions are evaluated using C precedence rules, which are indicated by the rows in Table 11. Operators with higher precedence are at the top of the table, and operators with equal precedence appear in the same row. Note, however, that the logical AND and OR operators do NOT short-circuit as they do in C; both operands are evaluated, regardless of the truth value of the first operand.

An expression that does not reference any labels or external symbols (declared with the `.declare` or `.extern` directive) is an absolute expression. There is one exception to this constraint on absolute expressions: any subexpression of the form `<label1> - <label2>`, where both labels are defined in the same segment within the current input file,

can be evaluated by the assembler and is therefore absolute. Further operations may be performed on such a subexpression, and it remains absolute, subject to the same constraints regarding labels and external symbols.

If a label defined in a non-absolute segment is used in an expression that does not fit the criteria for an absolute expression, that label must be declared as a global for the linker to resolve the expression (i.e., `.global <label>`).

The assembler evaluates absolute expressions. They are required in certain situations, such as the operand to the `.align` directive, where the assembler must be able to evaluate the expression.

A numeric expression that is not absolute is said to be relocatable. Relocatable expressions can use the operators in Table 11 on page 38, except for the following:

```
!     &&     ||     ==     !=     < >     <=     >=     ^     unary+
```

NOTE: CFCC evaluates the operators of a relocatable expression during the *linking* phase.

Symbols created using the `.equ` directive follow the same rules, described in the preceding paragraphs, for determining whether their values are absolute or relocatable.

Arithmetic operations use 32 bits; all operations are signed using two's complement representation. The right-shift operator `(>>)` is "arithmetic" when applied to a negative value. That is, high-order bit positions are filled with copies of the sign bit. The division operator `/` and remainder operator `%` perform integer division that truncates towards zero, for negative and positive operands. This behavior applies to both absolute and relocatable expressions.

5.3.3 Assembler Directives

The numerous assembler directives can be classified by functionality:

- Declaring Symbols
- Defining Data
- Changing the Program Counter
- Partitioning Memory
- Controlling Assembler Checking
- Preprocessing
- Controlling the Listing File
- Defining Special Object File Entries

5.3.3.1 Directives for Declaring Symbols

.equ <name>, <num-expr>

This directive gives a symbolic name to an expression. The `<num-expr>` can contain references to values that you define later, as long as all values are fully defined by the time the assembler starts processing the line in the second pass.

.declare <label-list>

This statement identifies labels that are defined externally, allowing subsequent assembly code to reference those labels. The `.declare` directive creates *weak* references to the labels; only those labels that are actually referenced cause the object file in which they are defined to be linked into the program by CFCC's linking phase. The `<label-list>` is a comma-separated list of labels.

.extern <label-list>

This statement identifies labels that are defined externally, allowing the program to reference the labels and forcing them to be linked with the program. The `.extern` directive creates *strong* references to the labels; they are linked into the program by CFCC's linking phase due to the `.extern` directive, regardless of whether they are referenced by the code. The `<label-list>` is a comma-separated list of labels.

.global <label-list>

This statement identifies labels that can be referenced in other source files. The `.global` directive for a label can either precede or follow the definition of the label, but it must appear in the file in which the label is defined for the label to be available in other source files via `.declare` or `.extern`. The `<label-list>` is a comma-separated list of labels.

5.3.3.2 Directives for Defining Data

.data <num-expr-list>

This directive evaluates the numeric expressions contained in the comma-separated `<num-expr-list>` and places the results in consecutive words of memory, with one major exception.

For all segment types except XY data memory segments, each `<num-expr>` generates a data word. Within an XY segment, each pair of `<num-expr>`s generates a 48-bit XY-memory "word", where the first `<num-expr>` in the pair is the high-order 24-bit value that goes into Y memory and the second `<num-expr>` (which defaults to 0 if there is an odd number of `<num-expr>`s) is the low-order 24 bits that go into X data memory.

.fill <num-expr-1>, <num-expr-2>

The directive emits words with value `<num-expr-2>` until the program counter has `<num-expr-1>` in its least significant bits. If, when the `.fill` directive appears, the program counter has the required least significant bits, then this directive emits no words and does not change the program counter.

`<num-expr-1>` must be an absolute expression evaluating to a positive integer. This integer places an alignment restriction on the current segment: if x is the smallest power of 2 that is greater than `<num-expr-1>`, then the starting address of the current segment is a multiple of x. If the current segment has an absolute address, then this address must also be a multiple of x.

`<num-expr-2>` must be an absolute expression. There are no restrictions on its value, but only the least significant *N* bits will be used, where *N* is the minimum of 32 and the wordsize of the current segment. For example, `.fill 13, 255` forces the current segment to start at an address that is a multiple of 16, and emits words with the value `255` until the least significant binary digits of the program counter are `1101`, which is the binary representation of `13`.

.zero <num-expr>

This directive sets the next `<num-expr>` entries in the current segment to zero. `<num-expr>` must be an absolute expression.

.bss <num-expr>

This directive reserves the next `<num-expr>` entries in the current segment, but does not assign any initial value to them. `<num-expr>` must be an absolute expression. The memory region is reserved but uninitialized in CFCC's output files (see Appendix A, "File Formats" on page 84), but the behavior of program loaders varies for such memory regions; some loaders clear `.bss` regions and some do not modify `.bss` memory regions.

5.3.3.3 Directives for Changing the Program Counter

.org <num-expr>

Use this directive to set the current program counter to <num-expr>, the value of which must be either absolute or relative to the current segment. In the expression, you must not refer to a constant name or label that is defined after the .org directive. Fully define all values used in the <num-expr> in the first pass before the .org directive is processed. The new origin is always relative to the start of the current segment.

.align <num-expr>

Sets the current program counter to the next address that is a multiple of <num-expr>. For example, .align 4 moves the program counter to the next four-word boundary, if the program counter is not already at that boundary. The <num-expr> must be absolute and a power of 2. The start address of the current segment in the current file is also forced to the desired alignment boundary. If more than one .align directive is in a segment in a file, the segment is aligned according to the maximum .align value found. If the current segment has an absolute address, then this address must be a multiple of <num-expr>.

IMPORTANT: If there is a label on the statement, its address is not aligned as you might expect, because the new alignment does not take effect until the beginning of the next statement.

For example, instead of:

```
label: .align 0x10          // the label will NOT be aligned
```

you should say:

```
    .align 0x10
label:                      // the label will be correctly aligned
```

5.3.3.4 Directives for Partitioning Memory

.pseg [<name>]

.pseg [<name>,] <option-list>

The .pseg directive creates a program-memory segment. The <name> defaults to “program” if it is not given. There are also .xseg, .yseg, and .xyseg directives that have similar syntax but differ in terms of the memory bank of the segment that they define or reference. Table 12 lists the memory-bank parameters and the default segment names for all of these directives.

Table 12. Segmenting Directives

Directive	Memory Bank	memtype	wordsize	Default Name
.pseg	Program memory	0	32	program
.xseg	X data memory	1	24	xdata
.yseg	Y data memory	2	24	ydata
.xyseg	X and Y data memories	3	48	xydata

The .pseg family of directives instruct the assembler to begin assembling code or data into a memory segment with the given or default name, which has characteristics as shown in Table 12, and as given by <option-list> if it is provided.

Memory segments allow the program's code and data to be grouped into logically related sections. These directives allow the programmer to specify which memory bank contains a given section of the program, and where, within that memory bank, the code and data are located.

The memory areas in an SK4 system are described in detail in the *Hardware Reference Manual for Ezairo 7100*. More information can also be found in Section 7.4, "Memory Configuration" on page 54.

You can use these directives with the same <name> multiple times within a module, or in different modules. If you specify the same segment multiple times throughout a program, provide the same option values each time in the <option-list>; the assembler and linker enforce this. After the first time that a given segment is used in a module, the short-form .pseg [<name>,] append can be used in the same module; the options all take the values previously specified. A similar short-form can be used with .xseg, .yseg, and .xyseg.

Segment names are treated as a separate class of names, so it is possible to have a label and a segment with the same name.

If no segment directive is given at the start of an input file, the assembler acts as if the directive .pseg precedes the code in the file.

The <option-list> is a comma-separated list of options, which are described in Table 13.

Table 13. Segmenting Directive Options

Option	Description
base=<num-expr>	Specifies an absolute base address <num-expr> at which a segment is to be linked. If there are several segments with the same memtype and base, they are arranged in order of link time appearance at successively higher addresses starting at or above the base. If previously linked segments were allocated in space up to and beyond the base, then the segment is allocated at the first available address above the base. If this option is not specified, no base address restriction is assumed.
abs=<num-expr>	Specifies that the segment is absolute; it must be linked starting at the address <num-expr>. All other segments are relocatable, which means that CFCC can place them anywhere in memory during the linking phase. Space for absolute segments is always allocated first by CFCC, during linking. CFCC generates an error message if an attempt is made to overlap two or more absolute segments. An absolute segment with a given name (or the default name) cannot be used in multiple modules.
align=<num-expr>	Specifies that the segment's start address must equal zero modulo the given <num-expr>, which must be a power of two. The default alignment value is 1.
rom	Specifies that the linker should locate the segment in an area of memory that has been designated as ROM using CFCC's --romN or --nromN option.
append	Specifies the option values provided in a previous definition of the segment. This option cannot be used with any other options. The segment must be previously defined in this module.

We recommend that you use these short-form segment directives, instead of the more general .segment directive, so that the memtypes and wordsizes of segments are consistent.

.segment <name> [, <option-list>]

The .segment directive is the most general way to declare a new memory segment. However, with .segment you can inadvertently create a segment that other Ezairo 7100 development tools cannot recognize (due to an incorrect memory bank number or wordsize). Therefore, we recommend that you use the .pseg family of directives instead of .segment. CFCC generates an error message when the .segment directive is used unless you give the --dotsegment option.

The use of segments is described above for the .pseg directive. The append option cannot be used with .segment; the corresponding short-form for .segment is simply .segment <name>.

The .segment directive accepts the options shown in Table 13 on page 42, with the exception of append, and additional options shown in Table 14.

Table 14. .segment Options

Option	Description
memtype=<num-expr>	Specifies the memory bank to which the segment corresponds. The valid values for <num-expr> are 0 (program memory), 1 (X data memory), 2 (Y data memory), and 3 (XY data memory). The default memory type is 0.
wordsize=<num-expr>	Specifies the number of bits in a word, in the segment. Program memory must be specified as 32 bits wide; X and Y data memory as 24; and XY data memory as 48.
common	Segments defined with the common option are handled specially during the linking phase. If common segments with the same name appear in different modules, CFCC assigns the same block of memory to all of them. All common segments with the same name must be the same size, and at most one can contain data; the rest must use only .bss directives to determine the size of the segment. These restrictions are enforced by the linker. By default, segments are not common.
blank_common	Segments defined as blank_common are similar to those defined as common, with two exceptions: • No blank_common segment can contain any data. • blank_common segments in different modules do not have to be the same size; the linker uses the largest size defined. These restrictions are enforced by the linker. By default, segments are not blank_common.

5.3.3.5 Directives for Controlling Checking

These directives are provided by the CFX assembler for controlling, on a per-instruction basis, modes that enable or disable various kinds of assembler checking. The setting for each mode (associated with each directive) is maintained in a stack, with the top-of-stack value controlling whether the mode is on or off at a given point in the compilation. Each stack is initialized with a single entry, based on CFCC command-line options.

The check-controlling directives take a single keyword parameter that determines the action taken by the directive, as follows:

ON	Turns the mode on by replacing the top-of-stack element with **On**.
OFF	Turns the mode off by replacing the top-of-stack element with **Off**.
PUSH	Pushes a new element onto the top of the mode's stack. The new element has the same value as the previous top-of-stack value.
POP	Pops the top element off of the mode's stack, so the next element down becomes the top-of-stack value and the current value for the mode changes to that value.

The checking modes are maintained in stacks so that separately-developed utility modules such as header files and macros can set the modes that their code requires, while maintaining the modes of the surrounding (invoking) code that are determined by another developer or team. The examples in the sections below illustrate the typical usage.

.strictnop <action>

The `.strictnop` directive controls whether `NOP` instructions are required (strict) or optional (not strict) with *Arithmetic-Move* instructions that only have an *Arithmetic* or a *Move* part. See Section 5.3.2.1, "Instruction Forms" on page 35 for more information about optional `NOP`s.

This mode is initialized to **Off** (not strict) by default. If the `--strictnop` option is given on the CFCC command line, this mode is initialized to **On** (strict).

A typical use in a `#macro` might be:

```
.strictnop push
.strictnop off
mul a0, x0, y0  // Arith with no Move NOP
.strictnop pop
```

.strict <action>

The `.strict` directive controls whether instruction synonyms and synthesized instructions are accepted by the CFX assembler. See Section 5.3.2.2, "Synonyms and Synthesized Instructions" on page 36 for more information about these instructions.

This mode is initialized to **Off** (accept synonyms and synthesized instructions) by default. If the `--strict` option is given on the CFCC command line, this mode is initialized to **On** (strict, do not accept synonyms and synthesized instructions).

.check <action>

The `.check` directive controls whether the assembler checks for restricted (invalid or dangerous) instruction combinations. See Section 5.2.2, "Instruction Restriction Checks" on page 31 for more information about restriction checking.

This mode is initialized to **On** (check restrictions) by default. If the `--no-check` option is given on the CFCC command line, this mode is initialized to **Off** (do not check restrictions).

If restriction checking is turned off for a *containing* instruction—a `LOOP` instruction, a control-flow instruction with delay slots, or an *Arithmetic-Move* instruction involving a multiplication (or two)—checks are disabled for that

instruction *and* all instructions that it “contains”: the instructions in the loop, the delay slot instructions, or the instructions that execute in the second cycle of the multiplication.

A typical use of `.check` might look like:

```
    // This code should be clean - turn checks on
    // (Often done at the top of the file, or left out
    // with the assumption that checking is on by default.)
    .check on
    ...
    cmp x0, y0 ||| nop
    goto.2d same_values if eq
    sub x0, x0, y0 ||| load y1, 5  // [delay slot 1]
    mul a0, x0, y0 ||| nop         // [delay slot 2]

    // This conditional GOTO is okay because Flag Updates
    // are disabled, so MUL does not affect the condition
    .check off
    goto.0d less_than if lt
    .check on
same_values:
    ...
```

The programmer has disabled restriction checks for the `GOTO.0D` instruction to suppress the error that the assembler would otherwise generate; it is safe to turn off checking on that instruction because the programmer knows that the Flag Updates mode is disabled (which the assembler cannot know). If the instruction at the label `same_values` violated a multiplication restriction, the assembler would catch this. If restriction checks were turned off for the `MUL` instruction instead, this would suppress the error message on both the `GOTO.0D` *and* the instruction at `same_values`.

5.3.3.6 Directives for Preprocessing

NOTE: It is generally preferable to use CFCC preprocessing directives, instead of comparable assembler directives. For more information, see Table 5 on page 22.

.error <text>

This directive causes a fatal error message, containing the given `<text>`, to be written to the standard output. It is preferable to use the CFCC `#error` preprocessing directive. The assembler aborts processing on the second pass, so the instructions and directives following this directive must be valid. No listing file is produced.

.warning <text>

This directive causes a non-fatal warning message, containing the given `<text>`, to be written to the standard output. It is preferable to use the CFCC `#warning` preprocessing directive.

.if <num-expr>

This directive begins a conditional assembly section. The `<num-expr>` cannot be a relocatable expression and may only contain values that have already been defined. If the value of `<num-expr>` is not zero, then the assembly code following this directive is assembled until the corresponding `.else` or `.endif` directive is found. You can nest conditional assembly sections. It is preferable to use the CFCC `#if` preprocessing directive.

.ifa <num-expr>,<min>,<max>

This directive begins a conditional code selection. The <num-expr> must be an absolute expression, but it may contain values that you define later in the file. The <min> and <max> must be absolute expressions, and may only contain values that have already been defined. If the value of <num-expr> is between <min> and <max> inclusive, the assembler emits the assembled code between this directive and the corresponding .else. If the value is outside the range of <min> to <max>, the assembler emits the assembled code between the .else and the .endif.

There are several important differences between .if and .ifa. The controlling expression of a .if must have a value that is known and fixed at the time the assembler reads it. At that point the assembler selects the appropriate part of the .if — either the .if part or the .else part — once and for all. It processes the instructions and directives in the chosen part, and completely ignores those in the other part. By contrast, the value of the controlling expression of a .ifa need not be known until the end of the file. The assembler reads and processes the instructions and directives in both parts of the .ifa. As a result, no single label or name can be defined in both parts. It also means that it is normally not a good idea to have directives such as .pseg in either part of a .ifa. However, .org and .align directives, as well as directives that define data words (namely .data, .string, .pack, .zero, and .bss), behave just like instructions, in that they are emitted only if the enclosing part is selected.

Once the assembler has read the entire input file, it determines the address of each label under the assumption that the controlling condition of every .ifa will be met — that is, that every <num-expr> will fall within the specified range. It then evaluates the controlling expression of each .ifa. If the controlling condition of some .ifa fails, then the assembler selects its .else part. Since the .else part need not contain the same number of instructions as the .ifa part, this may change the locations of some labels. So, the assembler repeats the process for each remaining .ifa, using the new values of labels.

The .ifa directive is designed to select between short and long versions of operations like branching or memory addressing. We recommend that the .ifa part contain the preferred (short) form, and the controlling expression indicate the condition under which the short form is valid.

.else

This directive begins a conditional assembly “else” section. If the .else corresponds to a .if directive whose <num-expr> is zero, the assembly code following this directive is assembled until the corresponding .endif directive is found. If the .else corresponds to a .ifa directive, the assembly code following this directive is selected if the .ifa condition fails.

.endif

This directive terminates a conditional assembly section.

5.3.3.7 Directives for Controlling the Listing File

.title <text>

The argument <text> consists of everything up to the end of the .title line. If listing is enabled, the <text> is displayed on subsequent page headings.

.eject

If listing is enabled, this directive causes the next line to appear on a new page.

.nolist

This directive inhibits the generation of the listing file.

.list

This directive re-enables the printing of lines to the listing file. You can nest the .list and .nolist directives, and they simply cancel each other. If more .nolist directives are processed than .list directives, listing is disabled.

5.3.3.8 Directives for Defining Special Object File Entries

For more information on using these assembler directives, see Section 5.4, “Generating Debugging Symbols” on page 49.

.comment <text>

This directive causes a comment record, containing the given <text>, to be written to the relocatable object file.

.file <filename>,<level>

This directive describes a file name table entry for debugging. The <filename> can be any double-quoted string. The <level> is a small integer that you can use to distinguish between different types of files. It can also be the keyword .file, meaning the level conventionally assigned to assembler input files (14), or .list, meaning the level conventionally assigned to assembler listing files (15). You can use any number in the range 0 to 31, but certain numbers have conventional interpretations or are used by particular tools as described in Table 15.

Table 15. File Levels

File Level	Conventional use
0	Unknown file type.
4	Reserved for C programs.
6	Reserved for C programs.
7	Assembler output, line numbers for #macro refer to invocation.
8	Original assembly code.
10	Preprocessed assembly code.
15	Listing file.

.line <line-number>, <filename>, <level>, <num-expr>

This directive describes a line number table entry, for debugging. Everything but the <line-number> is optional. The <line-number> can either be an absolute expression, or the keyword .line, meaning the current input file line number, or the keyword .list, meaning the current listing file line number. The <filename> and <level> are described under the .file directive. If the <filename> and <level> are omitted, the assembler uses the file most recently given in any .line or .file directive. Finally, the <num-expr> provides a segment-relative address to be associated with the line number; if omitted, the assembler uses the current address in the current segment.

.dbg <dbsym>, <infotype>, <info> [, <infotype>, <info>]

The .dbg directive defines or adds to an entry in the debug symbol table. Debug symbols are distinct from assembler symbols. Debug symbols may or may not directly correspond to assembler symbols. A debug symbol has a name and associated debug information. Each item of debug information consists of an <infotype> and corresponding

<info>. The <infotype> is a positive integer in the range 1 to 65534 that identifies a class of debugging information; for example, address, data type, storage class, etc. The assembler and linker impose no interpretation on <infotype> values in the allowable range, but conventionally, <infotype> 1 is used for a symbol link address. (The linker reserves the <infotype> values 0 and 65535 for the debug symbol name and debug symbol table index, respectively.) Normally, it is a good idea to use small <infotype> values because the default debug symbol table format generated by the linker encodes consecutive <info> values in <infotype> order, from 1 to the highest defined <infotype>.

Several different kinds of <info> can appear in the debug symbol table entry.

1. A relative expression, such as a symbol address can appear. This example records the value of assembler symbol `foo` (that is, the address of label `foo`) in <infotype> slot 1 of the debug symbol `foo`:

```
.dbg foo, 1, foo
```

2. An absolute expression, such as a constant encoding some information about the symbol, can appear. This example records the constant `23` in <infotype> slot 2 of the debug symbol `foo`:

```
.dbg foo, 2, 23
```

3. The keyword `.line`, which enters the current input file line number in the symbol table entry, or the keyword `.list`, which enters the current listing file line number can appear. This example records the current listing file line number in slot 4, and records the current input file line number in slot 6:

```
.dbg foo, 4, .list, 6, .line
```

4. A debug symbol name, enclosed in braces, can appear. The debug symbol is created if it does not already exist. This technique could be used, for example, to associate a debug symbol for a C structure tag with a variable of that type. This example records the debug symbol `bar` in <infotype> slot 7 of the debug symbol `foo`:

```
.dbg foo, 7, {bar}
```

   In the linker output, slot 7 of `foo` will contain the symbol table index of `bar`.

5. A string, enclosed in double quotes, can appear. This example records the string `23 skidoo` in <infotype> slot 8 of the debug symbol `foo`:

```
.dbg foo, 8, "23 skidoo"
```

   In the linker output, slot 8 will contain the string table index of `23 skidoo`.

6. A file name, enclosed in angle brackets can appear. This example records the file name `multiply.a` in <infotype> slot 3:

```
.dbg foo, 3, <multiply.a>
```

   In the linker output, slot 3 will contain the file name table index of `multiply.a`.

7. The keyword `.file`, which enters the current input file. This example records the current input file in <infotype> slot 5:

```
.dbg foo, 5, .file
```

   In the linker output, slot 5 will contain the file name table index of the current input file.

In the object file, debug symbols appear in the order in which they were created.

5.4 Generating Debugging Symbols

The assembler can automatically generate debug table entries for debugging assembly code if you use the `-g` option in CFCC. These entries are placed in the debug symbol, file name, and line number tables of the SK4 ELF executable file generated by the linker, in addition to any entries defined by `.dbg`, `.file`, or `.line` directives.

CHAPTER 6
Using the Object Librarian

6.1 Introduction

CFCC drives CFX-AR to produce library files. CFX-AR is a GNU-based librarian, which creates and manipulates libraries of object modules produced by the GNU toolchain It accepts object files produced by the GNU toolchain (using CFCC with the `-c` and --toolchain=gnu options) as input as well as metadata files associated with those objects. For more information about CFX-AR, refer to the publicly available documentation for the GNU `ar` program.

6.2 Ordering Object Modules in a Library

See Section 7.3, “Ordering Object Module Libraries” on page 53.

CHAPTER 7
Linking Object Modules

7.1 Introduction

By default, CFCC links together relocatable object code into executable code. The executable output file format is called Executable and Linking Format (ELF), and is described in Section A.2, “Executable and Linking Format (ELF)” on page 84.

7.2 Linking Relocatable Object Code

This section describes the various CFCC linking options. For information on CFCC syntax, see Table 2 on page 16.

7.2.1 Linking Options

Available options are listed and described in Table 16.

CFCC only performs the linking stage in the absence of the `-E` and `-c` options, which halt processing after the preprocessing and compilation stages, respectively.

Output file names are determined in the following manner:

- In the absence of the `-o` option, the output file is called *a.cfe*, unless only one input file is given, in which case the output file name is the same as the input file name with its suffix replaced by *.cfe*.
- When the `-o` option is given, the output file is given the name provided.

Relocatable object files are linked in the order in which they are found while command options are collected. Memory area options (`--memN`) are sorted in ascending order by start address and checked for address overlap with previous memory area options for the same memory type. All other options are collected and set up before linking begins. If an option (such as `--mapfile` or `--entry`) provides a value, the last option of that type found is the one that is used.

Table 16. Command Line Options

CFCC Option	Description
filename	Gives the name of a file to be linked, which can be an object file, a source code file, or a library file. A source code file is first compiled, and CFCC links the resulting object module. A library file named directly on the CFCC command line (i.e., *not* as an operand to the `-l` option) is treated simply as a collection of object files; *all* of the object modules are linked into the resulting executable file. To search a library and link only the referenced object modules, see the `-l` option. CFCC assigns addresses in order from left to right for files named on the command line, and from the start to the end of a library file. A file to be linked at a lower address should appear first on the command line. The linker looks for object metadata (*.orc*) files with the same name as the object in the objects directory. Do not specify this file as input. It is not mandatory for an object file to have an associated metadata file.
`--mapfile` **filename**	Writes a load map to **filename**. The format of the load map is discussed in Section 7.5, “Load Map Format” on page 55. If **filename** does not have an extension of *.map*, that extension is appended. Do not use this option if you are using `--map`.
`--map`	Writes a load map to **outfile***.map*, where **outfile** is the prefix of the name used for CFCC’s executable output. Do not use this option if you are using `--mapfile`.
`--entry` **name**	This option allows you to specify that the symbol **name** is the entry address of the program being linked. To allow for the case in which you need to link together code for more than one processor, you can use several `entry` options. Declare each **name** used (with `.global`) in a different memory. If no `--entry` option is given for a particular memory type, an entry address of zero is assumed.
`-l` **filename**	Gives the name of an object library created by CFX-AR or by concatenating object modules. Only those modules that are needed to resolve previous external references are linked. Library files are read in the order they appear on the command line, and each library is searched sequentially; CFCC decides whether or not to link a particular object module based on the symbols it has already seen. Thus the order of libraries on the command line, and the order of object modules within a library file, can be significant. If the **filename** is a relative name (i.e. it does not begin with a backslash or a forward slash or a Windows/DOS drive letter followed by a colon), then CFCC looks for it first in the current working directory. If it cannot find it there, then it looks in the directories (if any) given with the `-L` option, searching the directories in the order they were given on the command line, from left to right.
`-L` **dir**	This option adds the directory **dir** to the list that will be searched by the linker to find libraries.
`-f` **name**	This option creates an external reference to the symbol **name**, equivalent to one created by a `.extern` assembler directive, before any object files or libraries are read. You can use this option to force an object module containing a global definition for the symbol to be linked from a library. This option permits linking to take place entirely from libraries when no object files are given on the command line.

Table 16. Command Line Options (Continued)

CFCC Option	Description
`--memN` **SSSS**/**EEEE** `--romN` **SSSS**/**EEEE**	This option is used to specify an area of RAM or ROM memory in which the linking process can allocate space. Use `--rom` to create ROM. The `--mem` option creates RAM. `N` must be a number, which is taken as the memory type. Possible values for `N` are 0 (for program memory), 1 (X data memory), 2 (Y data memory), and 3 (XY data memory). **SSSS** and **EEEE** must be numbers; they are the starting and ending addresses, respectively. All numbers can be C-style decimal, octal, or hexadecimal constants. More than one area of memory can be specified for each memory type. Two areas of a given memory cannot overlap. If you do not specify any memory area options, CFCC assumes that each memory type has an address space of 64 Kwords of RAM available starting at 0.
`--nmemN` **SSSS**/**EEEE** `--nromN` **SSSS**/**EEEE**	These options produce the same results as the `--memN` or `--romN` option, respectively, except that CFCC first clears any previous memory specifications for the memory type. This feature allows you to override any preconfigured memory configuration information.
`--overlap` **seg1**/**seg2**	Allows the segments named **seg1** and **seg2** to use the same physical memory locations. By default the double use (or *overlapping*) of one or more physical memory locations—possibly through the use of XY memory or the HEAR Configurable Accelerator's interleaved memories that are mapped into X and Y memories—is detected by the linker's memory-checking phase, causing the link stage to fail. The `--overlap` option tells the memory checker to allow overlaps between two named memory segments, which will presumably be used at different times during run-time so that their use of the same physical memory locations will not cause run-time problems.
`--overlay` **segments**	This is a short form notation which achieves the same result as `--overlap` by allowing you to specify more than two segments, each separated with a slash (`/`). This forms a transitive relationship where each distinct pair of segments can be thought of as a single overlap.
`--no-memcheck`	Disables checking for overlapped use of physical memory locations; CFCC does not run the linker's memory-checking phase. By default, the memory checker ensures that XY memory segments do not use the same locations as X or Y memory and that segments using the HEAR Configurable Accelerator's interleaved memories do not use the same physical locations in X and Y memories twice.

7.3 Ordering Object Module Libraries

The order in which object modules are placed in a library can be very important, because CFCC searches modules in the order that they appear in the library. The order in which libraries are specified on the command line can also be very important, because the linking process searches the libraries in the order that they appear on the command line. As a result, if a symbol is defined multiple times, CFCC always uses the first definition encountered, rather than any subsequent definition(s).

7.4 Memory Configuration

Use the `--memN` and `--romN` options to establish any limits on the addressing ranges allowed in the memories in your system. If you do not provide any such options, then CFCC assumes that all memory types have a 64 Kword address range (from `0` to `0xFFFF`). If your memory address ranges fall outside of the default, you will have to use `--memN` options.

If your memory address ranges fall within the default limits, no memory configuration is necessary. However, it is a good idea to configure your exact limits to give CFCC the ability to detect when it has run out of memory to allocate.

NOTE: It is assumed that memory type 0 (`--mem0`) is used for program memory, memory type 1 (`--mem1`) is used for X data memory, memory type 2 (`--mem2`) is used for Y data memory, and memory type 3 (`--mem3`) is used for XY data memory.

7.4.1 Memory Allocation for SK4 Systems

The flags that are used to tell CFCC about the memory layout of the SK4 system are as follows:

```
--mem0 0x8000/0xDFFF
--mem0 0xDFE0/0xDFFF
--mem0 0x3000/0x3FFF
--mem0 0x4000/0x4FFF
--mem0 0x5000/0x7FFF
--mem1 0x0000/0x2FFF
--mem1 0x4000/0x67FF
--mem1 0x8000/0xFFFF
--mem2 0x0000/0x0FFF
--mem2 0xC000/0xFFFF
--rom0 0x0000/0x0FFF
--rom0 0xE000/0xEFFF
--rom1 0x3000/0x307F
--rom1 0x6800/0x6FFF
```

In this example, note that:

- Memory space 0 is program memory or PRAM; the `mem0` specifier designates 24 Kwords of PRAM starting at address 0x8000, ARM Cortex-M3 core memory-mapped control and configuration registers, the interrupt vector table, as well as the ARM Cortex-M3 core program memory, data memory and memory-mapped registers.
- Memory space 1 is X data memory or XRAM; the `mem1` specifiers designate the 12 Kword regular data memory area, and the XRAM-mapped data memory shared with the HEAR.
- Memory space 2 is Y data memory or YRAM; the `mem2` specifiers designate the 4 Kword regular data memory area, and the YRAM-mapped data memory shared with the HEAR.
- The rom0 specifiers are for the read-only Program ROM and the HEAR program memory, which are mapped into the Program memory space. The HEAR program memory is RAM, but is treated as ROM by the tools to separate it from the CFX PRAM, and to indicate that it contains HEAR microcode that the CFX cannot execute. For more information about the HEAR and related software tools, see the *HEAR Configurable Accelerator Reference Manual*.
- The `rom1` specifiers describe the areas of X memory space where ROM lookup tables reside.

However, in actual usage, it is not a good idea to give the flags exactly as shown above describing the whole system memory layout to CFCC, because:

1. Most programs never use many of the above sections of memory. There is no practical use for accessing the Program ROM section, and the usual method for accessing the control and configuration memory is by loading pointer registers with the address constants that are defined in `<sk4_hw.cfi>`.
2. Part of each memory space is usually reserved for ON Semiconductor utility software, such as the Bootloader, the EEPROM library, the debugger, and CTK. ON Semiconductor reserves the low 32 words of X and Y memory, and the high 1 Kword of Program memory, for these purposes.
3. The CFX supports reading and writing 48-bit values from XY memory, consisting of a word from each of X and Y memory. Many programs will set aside a portion of X and Y memory to treat as XY memory. In the example below, 256 words of X and Y memory (from 0x0020 to 0x011F) are set aside as XY memory using the `mem3` specifier, but you are free to change this partitioning. Take care that the XY memory region does not overlap with either the X or Y memory regions.

For these reasons, the recommended options for CFCC with the hardware are as follows:

```
--mem0 0x8000/0xDFFF
--mem0 0xDFE0/0xDFFF
--mem0 0x3000/0x3FFF
--mem0 0x4000/0x4FFF
--mem0 0x5000/0x7FFF
--mem1 0x0120/0x2FFF
--mem1 0x4000/0x67FF
--mem1 0x8000/0xFFFF
--mem2 0x0120/0x0FFF
--mem2 0xC000/0xFFFF
--mem3 0x0020/0x011F
--rom0 0xE000/0xEFFF
```

7.5 Load Map Format

This format is a GNU load map format which is described in publicly available documents.

7.6 Converting Binary Executables to Text Format

CFCC produces executables in the binary ELF format, but it is not easy for humans to read nor is it readily usable in simple data manipulation scripts. For times when you need a textual representation of data, use the CFE File Viewer as described in the *Integrated Development Environment User's Guide*. On the **Absolute Data** tab, the data lines have the following form:

```
M AAAA DDDDDDDD
```

where:

- `M` is the memory space, specified as one of the following:
  - `P` for program memory
  - `X` for X data memory
  - `Y` for Y data memory
  - `R` for non-volatile storage, such as EEPROM.

- `AAAA` is the address specified in hexadecimal.
- `DDDDDDDD` is the data, also specified in hexadecimal. If the memory space is narrower than 32 bits, the values are padded on the left with spaces.

Here is an example of a program consisting of a single NOP instruction in text format:

```
P 8000 000000
```

Alternatively, you can use the CFX-OBJDUMP utility to get more information about one or more object files in a text file format. CFX-OBJDUMP is based on the GNU binutil objdump. For more information, see the publicly available documentation for objdump.

CHAPTER 8
Non-Volatile Memory Management

8.1 Introduction

The Non-Volatile Memory Manager (NVMM), which is integrated with the IDE, supports creating and viewing loadable, non-volatile (e.g., EEPROM) images for the Ezairo 7100.

You can use the NVMM to generate bootable image files by providing a bootloader and a list of applications to be included in the image. NVMM arranges the sections of your applications for placement in non-volatile memory, and generates the appropriate structures for the Ezairo 7100 EEPROM file system. You can later load these image files to the Ezairo 7100 non-volatile memory using the download tool, NVMD (*nvmd.exe*).

NOTE: NVMM ignores applications' uninitialized memory regions (created with the `.bss` directive). Nothing is written to the non-volatile memory image for these sections, so they are not initialized by the bootloader.

Files formatted with the standard NVI format can be viewed using the NVI Manager, an IDE component based on the NVMM, which provides a comprehensive, graphical representation of the EEPROM and image layout.

For details on the Ezairo 7100 EEPROM file system, refer to the *Firmware Reference Manual for Ezairo 7100*. For details on the NVMD application, refer to Section 9.2, "Non-Volatile Memory Download Tool" on page 76.

8.2 Features

NVMM supports the following features:

- Flexible Layout Arrangement
- Optimization through Automatic Equivalent Section Detection
- Image/Application Encryption
- Non-Volatile Memory Data Upload (NVMU) through the IDE
- Layout Visualization through the NVI Manager

8.2.1 Flexible Layout Arrangement

NVMM by default automatically arranges the moveable sections of the layout (applications and their sections, and the bootloader). While you are not required to specify addresses for each section, you can override this feature by specifying an explicit address. The automatic arrangement method is designed to make efficient use of the available space in EEPROM by placing sections in contiguous memory wherever possible.

8.2.2 Optimization Through Automatic Equivalent Section Detection

When multiple applications are present in a non-volatile image file, NVMM attempts to identify sections that are equivalent with respect to their data, their destination in memory, and any file system specific attributes/fields they might possess. By identifying equivalent sections, NVMM can include a single instance of the section in the layout and update other references to reuse that instance, thereby reducing the overall size of the resulting non-volatile image.

The implementation of this feature is file system dependent. For the Ezairo 7100 EEPROM file system, NVMM only performs automatic section matching if the section NVMM is trying to match is allowed to be freely placed. As such, NVMM does not attempt to match sections which have a defined address (either directly, by using @**<address>**; or indirectly, by defining an address for the application containing the section). However, if you

explicitly use the `@auto` address specifier for a section, it enables equivalent section detection even if the containing application was explicitly placed.

8.2.3 Encryption Support

NVMM supports both Level 1 and Level 2 security in the Ezairo 7100 EEPROM file system:

- Level 2 security applies to applications and their respective program memory sections.
- Level 1 security adds an additional layer of security to the EEPROM image by encrypting the keys employed in Level 2 security as well as the bootloader.

For more detailed information on the differences between the two levels of security and their respective use cases, refer to the *Firmware Reference Manual for Ezairo 7100*.

By default, the non-volatile images created with NVMM are not secure. For more information on creating secure images using NVMM in the IDE, refer to Section 8.3.1.5, “Securing an EEPROM Image” on page 64.

8.2.4 EEPROM Image Upload

The NVMU tool allows you to upload data from the non-volatile memory of a device. The data is read using a user-specified CTK configuration. For more information about uploading the EEPROM memory data, refer to Section 8.3.2, “Non-Volatile Memory Data Upload Through the IDE” on page 65.

8.2.5 Graphical Layout View Using NVI Manager

The IDE contains the NVI Manager, which is based on the NVMM. In the IDE, *.nvi* files open in the NVI Manager by default.

NOTE: NVI Manager opens *.nvi* files in read-only mode. For information on how to change the layout or contents of a *.nvi* file, see Section 8.3, “Using NVMM”.

8.3 Using NVMM

In the IDE, you can create an EEPROM image using the project wizard. This generates an NVI file that can then be loaded into the non-volatile memory on your Ezairo 7100. You can also open the *.nvi* file for viewing by using the NVI Manager.

8.3.1 Creating an EEPROM Image Using the Project Wizard

You can build NVI files using the standard new **Ezairo 7100 Development Project** wizard. See the *Integrated Development Environment User’s Guide* for more information about using this wizard. It creates a project that uses the NVMM builder to create the properly formatted image file for you, using the applications and options that you specify. For a full list of available options, and an explanation regarding their purpose and proper usage, refer to the Section 8.4, “Options” on page 69.

8.3.1.1 NVMM Applications Settings

When you create new projects using the project wizard and modify the Applications settings for NVMM, we recommend checking the new project’s Applications settings before building the project; otherwise, the new project uses the previously modified project’s Applications settings. To see and change the Applications settings for NVMM, begin with the project wizard as described in *Integrated Development Environment User’s Guide*, then:

1. After creating a project and giving it a name, click the **Next** button instead of the **Finish** button.

2. Click **Advanced settings…**
3. Select **C/C++ Build** > **Settings**; under the Tool Settings tab, navigate to **NVMM** > **Applications** as shown in Figure 2.
4. Make any changes as required and click **OK**.

You can also modify these settings after you create a project; you just have to build it again.

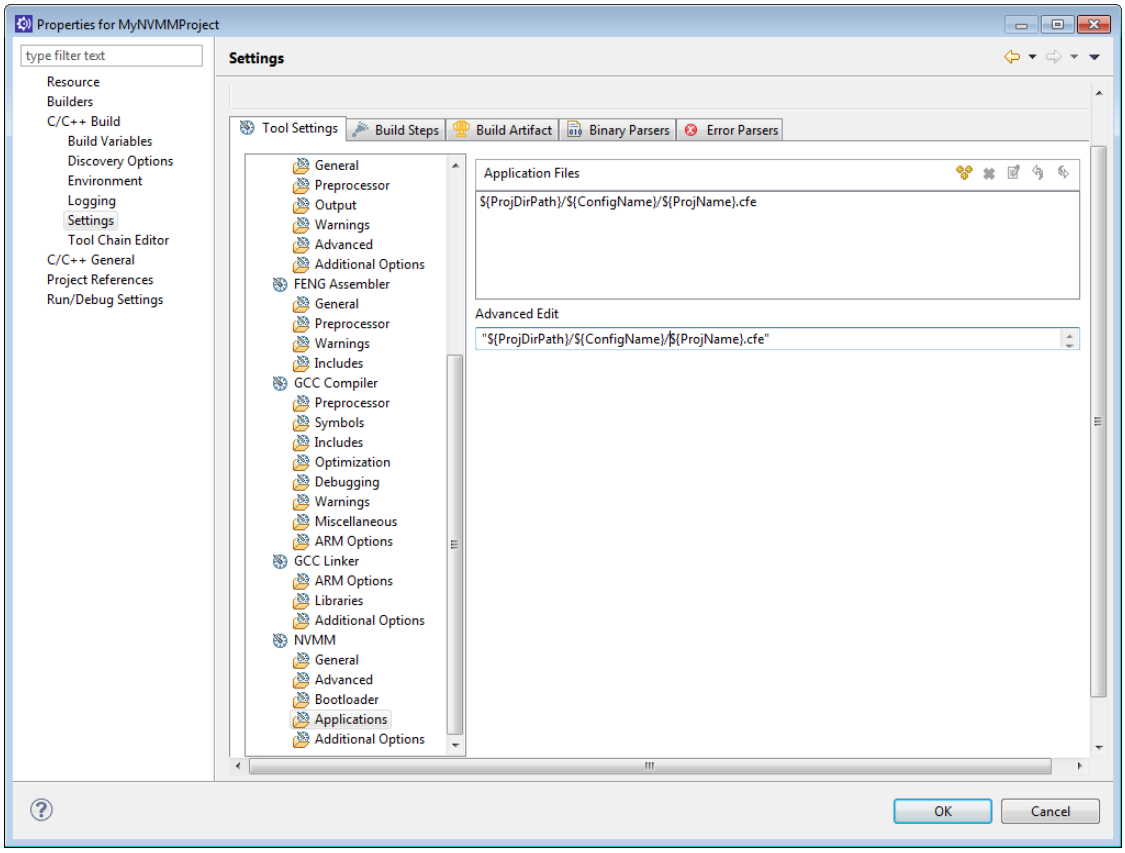


Figure 2. Applications Settings for NVMM Tools

8.3.1.2 Ezairo 7100 EEPROM Layout Defaults

The default Ezairo 7100 EEPROM layout possesses some characteristics that you must understand before attempting to transfer the layout to your device's EEPROM:

- Ezairo 7100 EEPROM file system *requires* a system area, key area, and a bootloader. Application files are optional and not needed to create a bootable EEPROM image.
- Layouts are not secure by default. That is, the default build settings for NVMM do not encrypt any of the applications or their respective program memory sections, the key area or the bootloader. Refer to Section 8.3.1.5, "Securing an EEPROM Image" on page 64 for more information on how to create secure images.

- By default, Ezairo 7100 EEPROM layout does not contain a manufacturing area, but reserves this area of EEPROM to prevent data from being placed in this address range. A manufacturing area can be created if required by loading the desired information from an arbitrary data file.

CAUTION: The manufacturing area contains device-specific calibration information. If you define your EEPROM layout such that it contains a manufacturing area, and you enable the overwriting of the manufacturing area by NVMD, any existing values in this area of EEPROM will be overwritten and therefore lost when the layout is transferred to the device. Make sure you back up your device-specific calibration values if you intend to supply your own manufacturing area (refer to the *Firmware Reference Manual for Ezairo 7100* for more information).

The IDE provides a read-only editor for viewing the contents of an EEPROM image file (*.nvi*), including a virtual view of the contents of EEPROM. To verify the content of your EEPROM layout, double-click your EEPROM image file in the IDE's Project Explorer view. It opens with this editor by default. For more information on viewing an image file in the IDE, refer to Section 8.3.4, “Viewing an EEPROM Image with NVI Manager” on page 67.

8.3.1.3 Editing Build Settings

By default, the IDE creates an EEPROM project containing build settings for the NVMM builder including initial values for all required components. To edit the build settings for your project:

1. Right-click your project in the Project Explorer view and select **Properties**. The editor opens to show the current build configuration.
2. Choose **C/C++ Build > Settings**.
3. On the **Tool Settings** tab, scroll down to select **NVMM** as displayed in Figure 3.

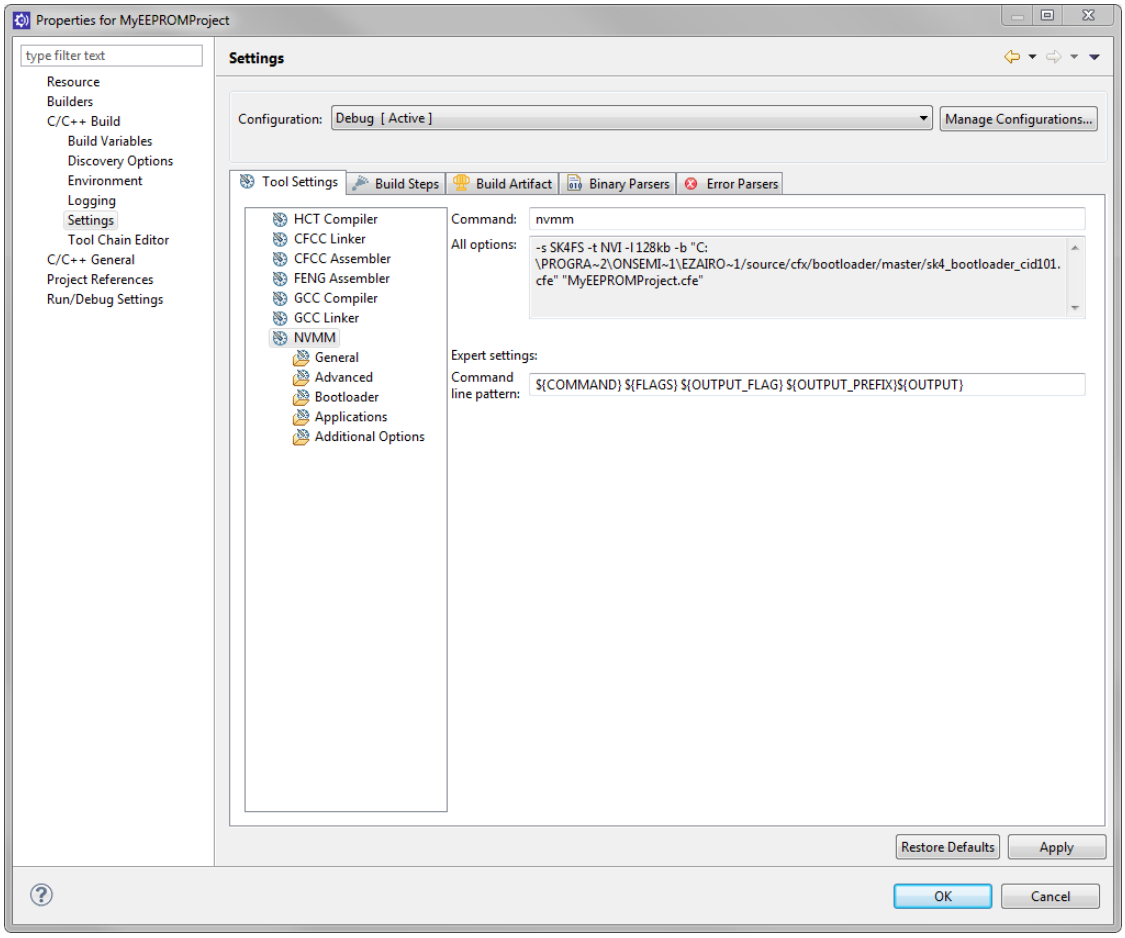


Figure 3. The General EEPROM Build Settings for a Project Build Configuration

4. Under **NVMM**, you see a list of available build settings grouped into categories that are described in more detail in Table 17. The build settings correspond to the available options for NVMM as described in Section 8.4, “Options” on page 69.

Table 17. NVMM Build Setting Categories

Field Name	Description
General	This category controls many of the settings for NVMM that affect the output (e.g., *Target image size*). Some of these settings are read-only, as they are determined by the project type (e.g., *Target file system*, *Image output*). For more information about the settings available in this category, refer to Section 8.4.1, “General Options” on page 70.
Advanced	This category contains many of the file-system specific options for NVMM. For Ezairo 7100, these include reserved blocks, field definitions, and loadable areas. Values for these settings can be defined by typing them directly into the provided text field. For more information on the settings available in the advanced category, refer to Section 8.4.2, “Global Field Options” on page 73, as well as Section 8.4.3, “Ezairo 7100 (SK4FS) Areas” on page 74.
Bootloader	This category deals exclusively with settings for the bootloader. The default settings contained in the controls will place the standard bootloader at the first available address (`0x180`). For more information on the available options and their usage, refer to Section 8.4.1.1, “Bootloader-Specific Options” on page 71.
Applications	This category allows you to specify the applications to be included in your output image. For more information on the options available in this category, refer to Section 8.4.1.2, “Application-Specific Options” on page 71.
Additional Options	This category contains a single text field for entering settings for options that are not available directly through any other build settings. Options and settings entered in this field are passed directly to NVMM.

When you finish editing the build settings and successfully rebuild the project, NVMM generates an EEPROM image file (*.nvi*) in the default build configuration output folder. The build console displays the status of the output. Refer to Section 9.2, “Non-Volatile Memory Download Tool” on page 76 for more information about transferring this image to your device’s non-volatile memory.

8.3.1.4 Adding an Application

You can edit the build settings for your new project and add a valid application to the EEPROM layout. To add an application, start by following the steps outlined in Section 8.3.1.3, “Editing Build Settings”, then proceed as follows:

1. In the **Settings Editor**, under the **Tool Settings** tab, select **NVMM > Applications** as shown in Figure 4.

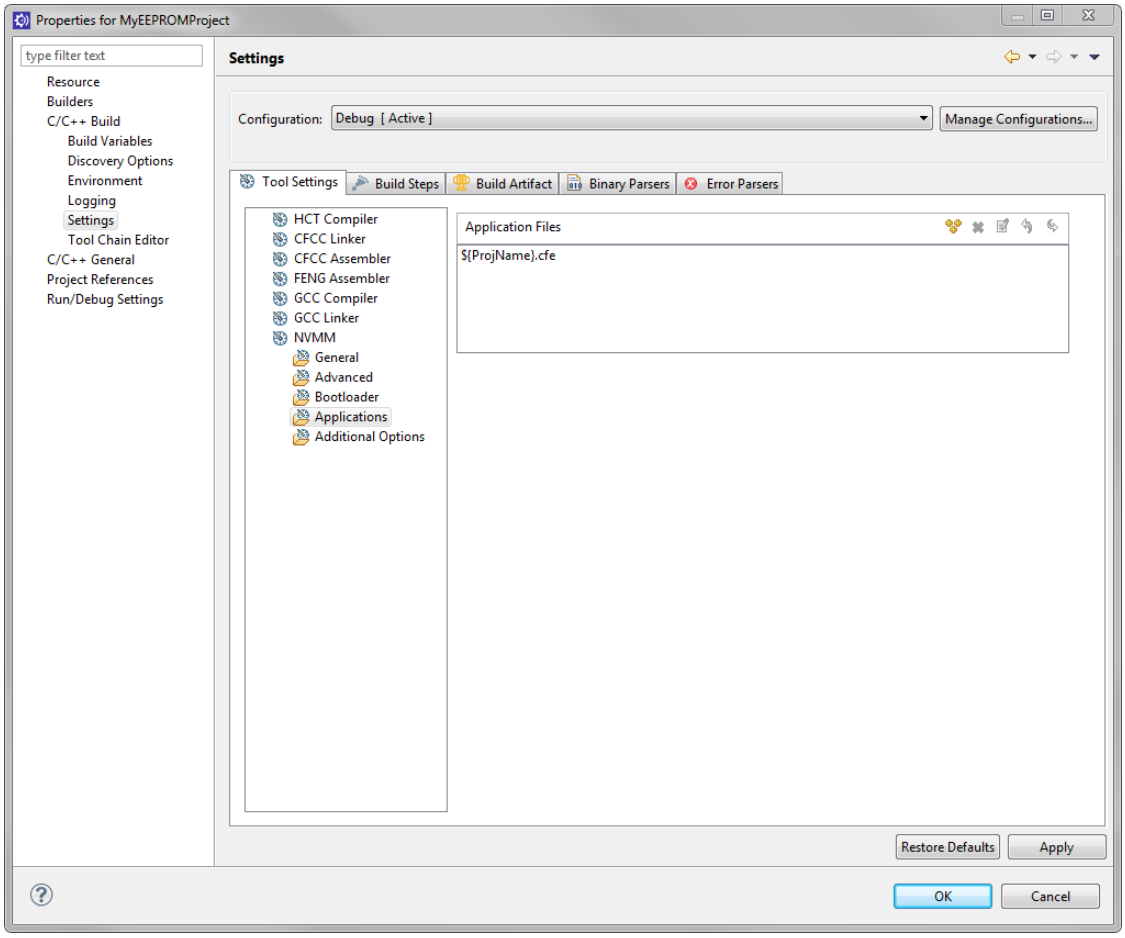


Figure 4. Adding Applications for an EEPROM Project in the Settings Editor

2. To add an application:
   a. Click the Add (+++) icon.
   b. In the open dialog box, browse either the workspace or the file system to the location of the application file (*.cfe* file type) and select it. If you are browsing the workspace, you might have to look in the Debug folder, which is the default build configuration output folder.
   c. Choose whether to encrypt the application or not. Alternatively, you can click the **Next** button and select specific segments to encrypt.
   d. Click the **Finish** button.
   e. Click **OK** to add the application file to the EEPROM project.

   The order in which the applications appear in the Edit Paths dialog determines the order in which they are added to the EEPROM image. If you have more than one application, you can use the Move Up and Move Down icons to change the order. Alternatively, you can use the `apptoload` field as described in Table 23 on page 74 to specify the index of the default application that will be loaded and executed by the bootloader. The default value is 1, corresponding to the first application in the list.

NOTE: If the application is part of the output from an existing project in the workspace, add a reference to the EEPROM project to ensure that build dependencies are satisfied. Adding a project

reference enforces a strict build ordering, ensuring that the EEPROM image is rebuilt automatically whenever the application is rebuilt. For more information on project references, see the section regarding inter-project dependencies in the *Integrated Development Environment User's Guide*.

3. Rebuild the project (right-click on the project and choose **Build Project** if the project does not build automatically). NVMM generates an EEPROM image file (*.nvi*) in the default build configuration output folder. The build console displays the status of the output. Refer to Section 9.2, "Non-Volatile Memory Download Tool" on page 76 for more information about transferring this image to your device's non-volatile memory.

8.3.1.5 Securing an EEPROM Image

The default build settings for NVMM produce unsecured EEPROM images. The implementation of non-volatile image security is file-system specific. To create a secure EEPROM image for Ezairo 7100, start by configuring your applications with Level 2 security:

1. Add your application as described in Section 8.3.1.4, "Adding an Application" on page 62. When configuring your application using the **NVMM Application Configuration Wizard**, select the **Encrypted** checkbox for the appropriate application and/or any applicable sections.

   For more information about the `encrypted` option, refer to Table 21 on page 73.

NOTE: By default, program memory sections inherit their encryption state from their parent application. If the application is encrypted, and no section-specific encryption is set, the section will be encrypted. Likewise, if the application is not encrypted, neither will the section be encrypted. To override the encryption settings for the application (i.e., to create an encrypted application containing one or more sections that are not encrypted or vice-versa), you must use an explicit, section-specific `encrypted` option.

2. Create a user key that NVMM can use to encrypt your application data. The user key is stored in the `user_key` field, in the key area of the EEPROM image. To specify a value for the `user_key` field in the IDE, edit the build settings as described in Section 8.3.1.3, "Editing Build Settings" on page 60, and select the **Advanced** category. Specify the value by entering it in the text field.

NOTE: If no value for `user_key` is defined, but an application or program section is declared to be encrypted with Level 2 encryption, the key area will be populated with randomly generated keys to ensure a bootable EEPROM image. The random keys are not stored, so the image will not be reproducible. If you want to create a reproducible, encrypted image, you must explicitly specify a value for the `user_key` field definition. This value will be stored in your EEPROM project build settings.

Your application data is now configured with Level 2 security. Level 1 Security builds on Level 2 security by encrypting both the key area used in Level 2 encryption and the bootloader in the non-volatile image output. To enable Level 1 security for the Ezairo 7100 EEPROM image:

1. Edit your build settings as described in Section 8.3.1.3, "Editing Build Settings" on page 60, selecting the **Advanced** category.
2. Specify a non-zero value for the `shared_key_index` field by typing directly in the text field. For more information on the `shared_key_index` field and its usage, refer to Table 23 on page 74.

NOTE: Level 1 security requires that the shared key corresponding to the selected index be installed on the developer's host PC to build the encrypted image. Keys are installed and managed through

the preferences found under **Window > Preferences > EEPROM > Security.** To install the key, click the **Install...** button to open a file browser, and select the keyfile provided by ON Semiconductor.

IMPORTANT: Level 1 security uses a secret, shared encryption key to encrypt data in non-volatile memory. ON Semiconductor assigns unique keys to individual customers. Contact your ON Semiconductor representative to request your key.

3. Save the build settings and build the project. (If auto-build is enabled, saving the changes to the build settings automatically rebuilds the image.) When the build completes successfully, your image is secure.
4. You can verify the image by opening the *.nvi* output with the NVI Manager. Refer to Section 8.3.4, “Viewing an EEPROM Image with NVI Manager” on page 67 for more information.

Encrypted images can be downloaded to a device in exactly the same manner as unencrypted images. Refer to Section 9.2, “Non-Volatile Memory Download Tool” on page 76 for more information on transferring this encrypted image to your device’s non-volatile memory.

8.3.2 Non-Volatile Memory Data Upload Through the IDE

Assuming you have an Ezairo 7100 containing a valid EEPROM image, you can upload the contents of the EEPROM and save the data as a text file. The IDE provides a menu command to connect to your Ezairo 7100 device and upload the EEPROM contents. To activate the command:

1. Select **Tools > Upload EEPROM to Image file...** which opens the upload dialog (similar to Figure 5).

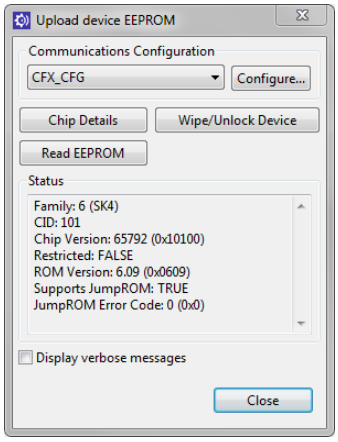


Figure 5. Upload device EEPROM Dialog

2. Select the Communication Toolkit (CTK) configuration to use via the **Communications Configuration** drop-down list. This specifies the communication configuration to use when connecting to your device.

NOTE: If you have not already created a configuration, or if you want to modify an existing configuration, click the **Configure...** button to open the CTK Configuration Manager. Refer to the *Integrated Development Environment User’s Guide* for details of creating or modifying a communication configuration.

3. If you have a valid communication configuration, you can click the **Chip Details** button to display basic information about the device to which you are connected. The **Status** window will show details such as Chip ID (CID), device family, ROM version, and whether the device is currently in restricted mode.

NOTE: If the device is in *restricted mode*, your attempt to read the device memory will fail and an error will be displayed in the status window. You must *unlock* a device operating in restricted mode before attempting to read from it. To unlock the device, click **Wipe/Unlock Device**. *Note that this erases any key data on the device*.

4. To enable verbose messaging in the dialog, click on the checkbox labelled **Display verbose messages**. This displays any communication details for the current session in the scrollable **Status** window, including each byte address and its raw contents (as a hexadecimal value) as read directly from the device.
5. Click the **Read EEPROM** button to connect to the device and upload the contents of the non-volatile memory.
6. Once the data has been successfully uploaded from the device, you are prompted with a Save EEPROM Image File dialog allowing you to select the location in which to save the new file. Navigate to a location on your file system and enter a file name. The default name is *eeprom_image*. If the file name that you enter exists in the selected location, you are prompted to allow overwriting.

IMPORTANT: If the device EEPROM contains encrypted data, the encrypted data is uploaded as it exists in non-volatile memory. Currently, decrypting encrypted data contained in images obtained from a device is not supported.

By default, the IDE preferences are set to automatically open a downloaded NVI file in the NVI Manager view. To disable this option, refer to Section 8.3.4.4, “NVI Manager Preferences” on page 69.

8.3.3 Reserving a Section of EEPROM

You can reserve an area of the EEPROM for special data, such as gain calculations, manufacturing data, or a serial number, starting at address 0x180. Depending on whether you want to load data in the reserved block, or leave it blank, choose one of the following procedures.

Leave the reserved area blank to fill with data later

This method creates a single EEPROM image file that you can load onto all EEPROMs, and then you can write the data to the EEPROM in that reserved section separately (either before or after the EEPROM image is written).

1. Right click on the project and choose **Properties**.
2. Expand **C/C++ Build** and choose **Settings**.
3. On the **Tool Settings** tab, scroll down to **NVMM** and choose the **Bootloader** category.
4. Select the **Advanced** category, and edit the **Reserve blocks** field to add a new reserved block. For example, to reserve space for a data table containing 32 bytes of data, the block might start at 0x180 and end at 0x200.

NOTE: The reserved range does not include the end address; in the example, 0x200 is the next free address following the reserved block.

Once you have saved and built your NVI project, you can open the EEPROM image file as described in Section 8.3.4, “Viewing an EEPROM Image with NVI Manager” on page 67. Notice that the section 0x180 to 0x200 contains no data and the bootloader is placed automatically at the next available address.

Put data in the reserved area at the time that you reserve it

This method requires you to recompile your NVI project whenever you change the data in the reserved section. If you update the data file and rebuild, the new calibration data is included in the image. For details about creating a reserved section of EEPROM with a data file, refer to Section 8.4.4, “Loading Arbitrary Data Files” on page 74.

8.3.4 Viewing an EEPROM Image with NVI Manager

The NVI Manager provides a read-only editor view for files with a *.nvi* extension within the IDE. To open an NVI file, do one of the following:

- In the Project Explorer view, double-click on a *.nvi* file.
- In the Project Explorer view, right-click on a *.nvi* file and select **Open With > NVI Manager**.
- Select **File > Open File...** and navigate to the desired *.nvi* file.

The NVI Manager interprets the NVI file format to render both a tree-view of the EEPROM file system and a graphical representation of the physical EEPROM layout. In addition to the NVI Manager view, additional representations of the NVI file can be selected by using the tabs located on the bottom of the view. Some aspects of the NVI Manager’s appearance and behavior can be configured through user preferences.

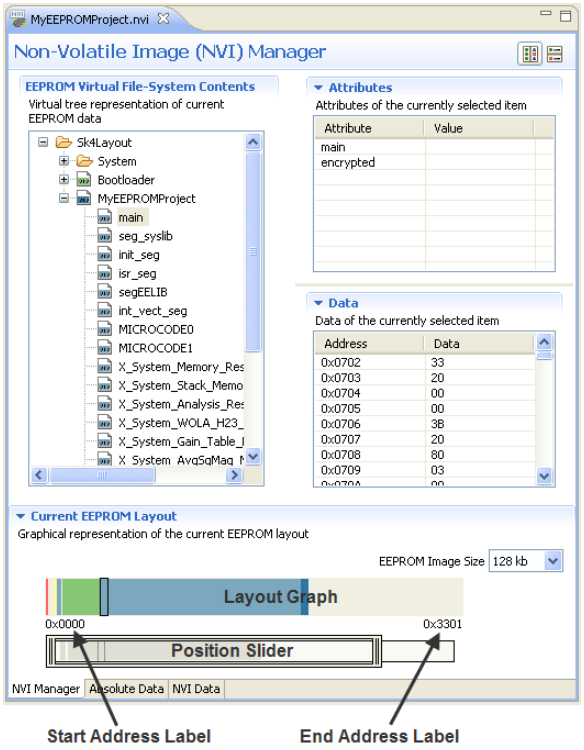


Figure 6. NVI Manager View

8.3.4.1 Virtual File System (VFS) Tree View

The tree view located in the top half of the NVI Manager provides the main interface for user interaction with the rendered layout. Tool tips are provided for all nodes in the tree, displaying the memory type and address where applicable. Selecting a node displays any attributes or data associated with it in the area to the right of the tree in their

respective expandable sections. If the node is associated with data, the selected node is also highlighted in the graphical layout at the bottom of the NVI Manager. If a node contains encrypted data, it is displayed with a *locked* icon representation.

The data associated with nodes can be exported to the system clipboard for use outside of the IDE. To export data:

1. Right-click on the node containing the desired data.
2. Select **Copy Data to Clipboard** from the pop-up menu.
3. The data, now on the system clipboard, can be pasted into other applications where supported.

NOTE: The **Copy Data to Clipboard** action is only supported by nodes associated with physical data.

8.3.4.2 Graphical EEPROM Layout View

The expandable section in the lower portion of the NVI Manager holds the layout view, which provides a graphical representation of the EEPROM as it would be after loading the current NVI file onto the device. Controls that can help you gain a clearer view of the data layout are labeled on Figure 6 on page 67 and described below:

Layout Graph

This horizontal color-coded bar provides a visualization of the physical EEPROM layout as it would appear on the device. The EEPROM contents are arranged against a blank image; the size of the image is determined by the **EEPROM Image Size** selector. The colored bars represent physical data in areas of memory defined by the file system specification and the NVMM builder options. You can select alternate colors through user preferences.

The start and end address labels define the memory range in view. These labels are automatically updated in response to the actions of the other controls to always reflect the range of memory in the current view.

The image responds to your mouse movements by providing tool tips containing the name, memory type and address of the node associated with the data at the specified point. The boundaries of the node are shown when the tooltip is displayed, and double-clicking on a colored area outlines the associated node. By default, selecting a node in the layout graph centers it (if necessary in a zoomed image), and selects it in the virtual file-system tree view. You can disable the option to snap to the tree selection in the user preferences.

Position Slider

This horizontal slider allows you to change the memory range in view when viewing a zoomed image. By default, this control is disabled because images are not zoomed when first opened. To enable the position slider, increase the zoom factor using the **Zoom Scale**.

The position slider shows the full EEPROM layout in the form of a grey bar that highlights the range of memory covered by the current image relative to the total size of the EEPROM. The slider itself delimits the memory range in view.

When viewing a zoomed image, you can move the slider left or right to show magnified regions of the EEPROM layout.

EEPROM Image Size Selector

This drop-down list allows you to specify the size (in Kbits) of the EEPROM that you would like to render your image against. This can be useful for determining memory usage on the

device. Selecting a different size scales the image against the new layout. The available selections are determined by the file system specified in the NVI file.

If you select a size that is too small for the EEPROM image, a warning symbol appears next to this field, and the graphical view is not refreshed until you select a valid size.

8.3.4.3 Additional NVI File Views

Besides the default NVI Manager view, the multi-page NVI editor features additional representations of the NVI file that can be useful when examining the contents of the file or troubleshooting potential problems with the image. The additional views include:

Absolute Data Represents the NVI file in absolute data format composed of three columns: memory space, address and data. For example:

```
...
R 0101 E5
R 0102 3C
R 0103 00
...
```

NVI Data Represents native NVI data starting with an overview of the VFS node hierarchy (if it exists) followed by a listing of the basic block structure and the data chunks that comprise the actual EEPROM image.

8.3.4.4 NVI Manager Preferences

A number of user configurable options allow you to customize the appearance and behavior of the NVI Manager. All preferences are accessed in the IDE through selecting **Window > Preferences...** in the main menu of the window, which opens the Preferences dialog. The preferences associated with the NVI Manager, described below, are located under EEPROM.

Automatically open uploaded EEPROM images (NVI format)
This option automatically opens any uploaded file with a *.nvi* extension in the IDE with the NVI Manager. It is selected by default.

NVI Viewer Colors
Located under **NVI Manager**, this group of color-field editors enables you to customize the colors used for each component area of the image comprising the graphical representation of the EEPROM layout.

Snap EEPROM layout view to tree selection
Located under **NVI Manager**, this option synchronizes the graphical layout view with the current selection in the tree view. Selecting a new node in the tree moves the slider position to bring the selected area into range (centered, where possible), and highlights it. This can be useful when viewing large, zoomed images with many sections. This option is enabled by default.

8.4 OPTIONS

This section explains the available options for NVMM.

8.4.1 General Options

In its simplest form, NVMM constructs an image by providing it with a bootloader and one or more applications (up to 8 are allowed for Ezairo 7100). NVMM takes general options as well as both bootloader-specific and application-specific options. Table 18 describes the general options available for NVMM.

Table 18. General Options

Option	Name	Description
`-h`	Help	Passing this option causes NVMM to output the default help information (which is given also when NVMM receives an invalid option, or no parameters) along with file-system specific help. For more file-system specific information refer to the *Firmware Reference Manual for Ezairo 7100*.
`-v`	Verbose Mode	By default, NVMM operates in quiet mode and only displays warnings or errors. To see more information about the image build process, pass `-v` to NVMM. In this case, NVMM operates in verbose mode, and displays information about the progress of the build and how the image is being built.
`-w` **`workdir`**	Working Directory	This option is used to set NVMM's working directory. The working directory is used to calculate the path to a file when a relative path name is given. If this option is not specified, the working directory defaults to the current directory when NVMM is launched.
`-s` **`filesys`**	File System Type	In this release, the only file system supported is `SK4FS`, the file system used with Ezairo 7100, which is currently the default value for this option.
`-t` **`imagetype`**	Output Image Format Type	The only output image format type currently supported by NVMM is the NVI file format, which is the default value for this option.
`-l` **`size`**	Output Image Size	Use this option to specify the maximum image size that NVMM is allowed to generate. The size refers to the actual raw data that will be written to the non-volatile memory of the device, and not to the actual size of the image file. The size of the image can be specified in bits, bytes, Kbits, and Kbytes by using a number followed by `b`, `B`, `kb`, or `kB` respectively. NVMM aborts with an error message if the image size is exceeded.
`-strip`	Strip Debug Info	By default, NVMM uses the debug symbols obtained from any applications specified as input (must be valid *.cfe* files) to provide user-readable names for applications and sections in the image output. Use this option to create an image which does *not* contain information obtained from debug symbols. In this case, application and section names are automatically generated using default naming conventions.
`-o` **`output`**	Output File Path	Use this option to specify the path and file name that NVMM should use for generating the output image file. If this option is unspecified, NVMM generates a file with a default name (*i.nvi*) in the working directory.
`-r` **`addr-addr`**	Reserve Address Range	Use this option to prevent NVMM from placing any data in the specified address range. The range specified includes the start address, up to but not including the end address.

Table 18. General Options (Continued)

Option	Name	Description
`-f` **`field=x`**	Set Global File System Field	Sets the specified global field to the given value **`x`**. For more information on the available global fields and the range of valid values, refer to Section 8.4.2, “Global Field Options” on page 73.
`-d` **`area=f`**	Load File System Area	Loads the given file-system specific area with the contents of the given file **`f`**. The specified file must be a binary file of raw bytes, or a relative data file (ASCII) as described in Section 8.4.4, “Loading Arbitrary Data Files” on page 74. The contents of the file are copied byte for byte into the given area. For more information regarding the list of available areas, refer to Section 8.4.3, “Ezairo 7100 (SK4FS) Areas” on page 74.

8.4.1.1 Bootloader-Specific Options

You can specify bootloader-specific options appended to the bootloader filename. NVMM supports the bootloader-specific options described in Table 19.

Table 19. Bootloader-Specific Options

Option	Name	Description
`@address`	Address	The address at which the bootloader is placed in non-volatile memory. If unspecified, NVMM calculates an address automatically. See Table 20 for more information on how this address is calculated.

The Settings Editor in the IDE allows you to add the optional address at the end of the bootloader file path. For Ezairo 7100, the default value for the bootloader option already points to the compiled Ezairo 7100 bootloader for the appropriate device version; the compiled bootloader executables are included with the EDK and are found in **`<EDK_HOME>`***source**cfx**bootloader**master**sk4_bootload_cid101.cfe*.

8.4.1.2 Application-Specific Options

You can also specify application-specific options appended to each application filename. NVMM supports the application-specific options defined in Table 20.

Table 20. Application-Specific Options

Option	Name	Description
`@`**`address`**	Address	The application is placed at this address in non-volatile memory. If the address option is unspecified, NVMM automatically places the application and its sections in the layout according to the file system's specification. Specify the address as a numeric value in one of the C-style hex, octal, binary or decimal formats, or using the literal value **`auto`**. The implementation is somewhat file-system specific, but the general guidelines are: • If an address is defined as a number, the application and its sections are placed sequentially starting at the given address. • If the address is defined as **`auto`** (or unspecified), the application and its sections are automatically positioned at the first available location in memory (i.e., each section is positioned independently). For application sections (see the `!`**`section`** option below), you can also specify the `@`**`address`** option. It defines the address at which the specified application section will be placed. If this address is a number, the section is placed at the given address. If this address is defined as **`auto`**, NVMM automatically places the section regardless of the section's parent application address (which is useful for equivalent section matching). If the `@`**`address`** option is not specified for an application section, the section is placed automatically according to its parent application's address.
`#`**`field`**`=`**`x`**	Set Application Field(s)	Use the hash character (#) to specify an application-specific field (for a list of application-specific fields refer to Table 21), and set it to value **`x`**. You can define multiple **`field=x`** pairs, each starting with the hash character.
`!`**`section`**	Define Section-Specific Options	Use the bang character (`!`) to define section-specific options within an application. Section-specific options take the same syntax as application-specific options, but you must prefix those options with `!`**`section`** where **`section`** is the symbolic name of the section within the application (and remember that it is case-sensitive). For example: `app.cfe!main@0x2000#encrypted=true` The snippet above forces the `main` section in application `app.cfe` to be located at address `0x2000` in non-volatile memory and forces it to be encrypted. You can define multiple section-specific options simply by repeating the `!`**`section`** suffix. Also note that `~apptable` is a special application section specific to the Ezairo 7100 EEPROM file system referring to the application's vector table. See the *Firmware Reference Manual for Ezairo 7100* for more information.

When specifying explicit addresses for program memory sections that are encrypted, ensure that you allow for the additional padding (up to 3 bytes) that might be added to the section during encryption. For more information on encryption, refer to the *Firmware Reference Manual for Ezairo 7100*. NVMM will fail to create the non-volatile image output if sections overlap. When using automatic addressing, the extra padding is automatically taken into account.

NOTE: By default, NVMM processes the application sections corresponding to the segments in the linker output. Using the `!section` specifier for a named segment within an application file (with or without additional section-specific options) creates a separate EEPROM section for that segment. This can be used to arrange segments independently. There is however, overhead in the form of additional header blocks and calculated CRC values created when sections are split. In the case where the named segment occurs at the beginning or the end of an existing section, splitting results in the creation of two sections: one for the named segment and one for the remaining data. In the case where the named segment occurs in the middle of an existing section, the result of the split is three sections: one section containing the named segment, one section for the data that precedes it, and one for that which follows.

NVMM does not currently define any generic, application-specific fields, but the following application-specific fields are available for Ezairo 7100 (`SK4FS`) images:

Table 21. Ezairo 7100 (SK4FS) Application-Specific Fields

Field Name	Description
`encrypted=`**`boolean`**	Set the encrypted state of the specified P-section(s). Encryption implies Level 2 security (see the *Firmware Reference Manual for Ezairo 7100* for details). User keys, used to encrypt the section data, are defined using global fields as described in Table 23 on page 74. If no user keys are defined, NVMM automatically generates random keys to ensure a bootable EEPROM image is created.
`volatile=`**`boolean`**	Set the volatile state of the specified X- or Y-section. A volatile section type indicates that a non-critical error is generated when the CRC does not match the data, and the boot process continues even though the data in the section might be corrupt.

The Settings Editor in the IDE provides a convenient way to add most common, application-specific options described in Table 20. To configure application-specific options:

1. Right click on the project name and choose **Properties**.
2. Expand **C/C++ Build** and choose **Settings**.
3. On the **Tool Settings** tab, expand **NVMM** and select the **Advanced** category.
4. To configure common options, type the option settings directly into the text fields.
5. Click the **Apply** button.
6. Close the dialog box with the **OK** button.

To set options that are not available in the **Advanced** section, choose **Additional Options** from beneath **NVMM** on the **Tool Settings** tab, and type them directly into the text field.

8.4.2 Global Field Options

Table 22 describes the global fields available for setting in NVMM with the `-f` option.

Table 22. Generic Global Fields

Field Name	Description
`copyright=`**`string`**	Set the copyright information in the output image file.
`description=`**`string`**	Set the description in the output image file.

In addition to the global fields described in Table 22, you can also set the global fields specific to Ezairo 7100 as described in Table 23.

Table 23. Global Fields Specific to Ezairo 7100

Field Name	Description
`spispeed=`**`speed`**	Set the SPI port speed flag, where **`speed`** is one of `123kbps` (or `slow`) or `980kbps` (or `fast`).
`shared_key_index=`**`index`**	Set the shared encryption key index for Level 1 Security. The default is `0`, which means no Level 1 Security encryption. For Ezairo 7100, valid ranges for the Level 1 shared key index are 1-127 inclusive.
`apptoload=`**`number`**	Set the application number to load (default: `1`). For Ezairo 7100, the valid range is 1-8 inclusive.
`user_key=`**`key`**	Set the (128-bit) user key to **`key`** for encrypting P-sections (same as `user_key_entry1`). The value entered in this field is stored in the key area EEPROM section of your layout as required by the Ezairo 7100 EEPROM file system (`SK4FS`).

8.4.3 Ezairo 7100 (SK4FS) Areas

Table 24 describes the specific areas of Ezairo 7100 available for loading with the `-d` option.

Table 24. Ezairo 7100 (SK4FS) Areas

Area Name	Description
`mftrarea`	Load the entire manufacturing area (EEPROM byte addresses `E:0x0020` to `E:0x007D`). If the file contains less than the 94 bytes of data required to fill the manufacturing area, the area is padded to the end with `0x00`. The CRC value is automatically calculated on the entire area once the data is loaded from the file. If the file is larger than 94 bytes, only the first 94 bytes are used. The CRC is calculated on the data contained in those 94 bytes and the rest is truncated.

8.4.4 Loading Arbitrary Data Files

To load arbitrary data into a specific region of EEPROM (e.g., for a fitting table or other set of application-specific values outside of an application), specify your data file as another “application” file in your NVMM settings.

The specifications for loading data files are as follows:

- The data file must have the extension *.d*.
- The file must be formatted as a relative data file containing a single column of data with one byte per line, in hexadecimal format (with or without a leading `0x`). For example:

```
00
01
02
```

- The file will be loaded into EEPROM at an auto-allocated address unless you explicitly specify an address with the `@address` suffix.
- The file will be loaded into your layout directly without modification. A CRC *will not* be calculated or appended to the data table.

NOTE: If you require a CRC or a checksum for the purposes of data validation, you must calculate it yourself and include it as part of the data table.

- The data file is *not* considered an application and *will not* be referenced in the system area.

NOTE: You cannot specify applications from raw data files.

CHAPTER 9
Non-Volatile Image Tools

9.1 Introduction

Tools that are available for managing non-volatile images are:

- The Non-Volatile Memory Download tool
- The Non-Volatile Image Dump tool

9.2 Non-Volatile Memory Download Tool

The Non-Volatile Memory Download (NVMD) tool is a light-weight tool for transferring the data in non-volatile image (NVI) files to the non-volatile memory of your device. NVMD uses the CTK to connect to the device, stop the system's DSP, perform a fast transfer directly to the device's EEPROM, and then initiate a system reset before disconnecting.

9.2.1 Using NVMD Within the IDE

This section describes using NVMD from within the IDE's graphical user interface.

9.2.1.1 Downloading a .nvi File

The IDE provides a graphical interface for transferring NVI files to the non-volatile memory of your device. After you successfully build Ezairo 7100 EEPROM project, you will have a *.nvi* file in the output folder of your project's default build configuration. This file contains an EEPROM image that can be downloaded to the EEPROM of your Ezairo 7100 device using NVMD. If you need more information about building an NVI file, refer to Chapter 8, "Non-Volatile Memory Management" on page 57.

To download a file to your device's EEPROM:

1. Locate the *.nvi* file in the Project Explorer view, which you can find in the Ezairo 7100 Development perspective. Right-click on it to display the context menu shown in Figure 7.

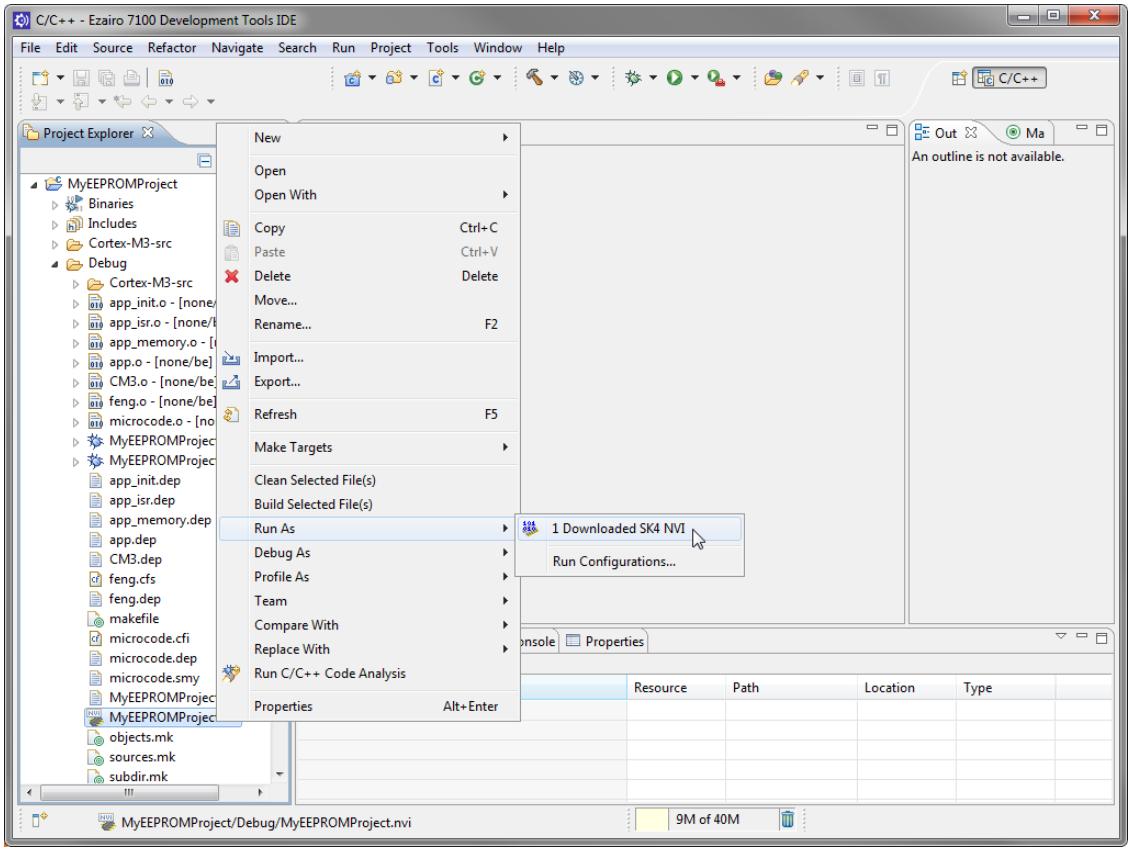


Figure 7. Menu for Downloading an EEPROM Image Using NVMD

2. Select **Run As** > **Downloaded SK4 NVI**.
3. If you have not downloaded the image before, you are prompted to select a communications configuration via the Communication Toolkit (CTK) Configuration Manager. If you have not created a communication configuration before, you need to add one now (refer to the *Integrated Development Environment User's Guide* for details about creating a communication configuration).
4. If the device is in restricted mode, you are prompted to wipe and unlock the device to gain access. Note that unlocking the device erases the key data. If you click **Yes**, the device is wiped and unlocked.
5. At this point, the IDE automatically launches NVMD as an external process; any output from the tool is sent to the Console view. When the operation finishes successfully, the image file has been downloaded to the device's EEPROM.

In the process of launching NVMD, the IDE creates and stores a launch configuration using the name of the NVI file by default. A launch configuration contains all of the settings necessary to relaunch NVMD exactly as it was the first time. This launch configuration contains various settings that control the execution of NVMD. More information on these settings is available in Section 9.2.1.2, "NVI Download Launch Configurations" on page 77.

9.2.1.2 NVI Download Launch Configurations

Launch configurations determine the context of an NVMD download operation. They specify which image file to download, and which communication configuration to use. They also store additional settings that control the execution of NVMD. Everything pertinent to an NVMD download operation is stored in its launch configuration.

You can view and edit launch configurations through the launch configuration dialog (shown in Figure 8), which can be displayed by selecting the **Run Configurations...** menu item that appears in several menus:

- Under **Run** in the main menu
- In the context menu when a *.nvi* file is selected in the Navigation view (right-click, choose **Run As**)
- In the toolbar, under the down arrow for the Run icon

In the Run Configurations dialog, under **Downloaded SK4 NVI** in the left pane, select the launch configuration that was created when you performed the procedure in Section 9.2.1.1, “Downloading a .nvi File” on page 76. The launch configuration has the same name as the NVI file by default. The **Image to download** field is already filled in, and you can change the options as necessary.

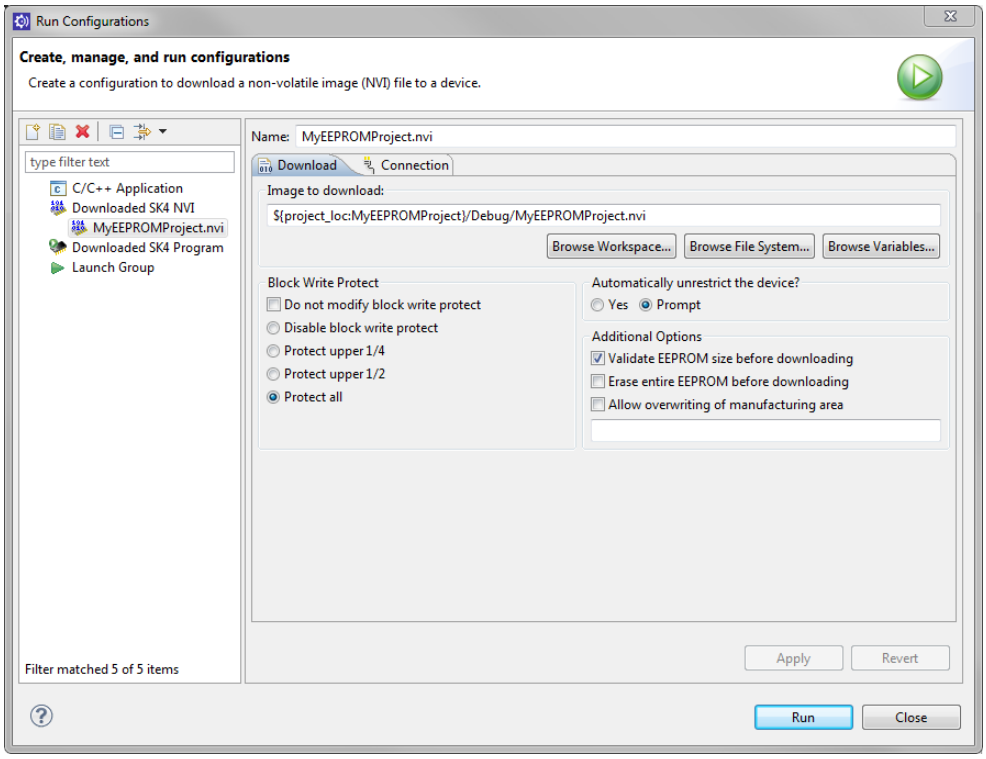


Figure 8. NVMD Launch Configuration Download Tab

The pane on the left side displays all the existing launches grouped by type. Configurations can be added and removed by clicking on the **New** or **Duplicate** and **Delete** buttons above it.

The right pane consists of two tabbed pages:

Download This tab identifies:

- The image file to download
- Whether to set the block write protect value to a specific value
- Whether to automatically unrestrict the device
- Whether to validate the EEPROM size before downloading

- Whether to wipe the device before downloading
- Whether to allow overwriting the manufacturing area of the device
- Additional command line options as specified in the space provided

Connection The communication configuration to use for the download operation.

9.2.1.2.1 Download Tab

The **Download** tab, shown in Figure 8 on page 78, is used to specify parameters related to downloading image files. The **Image to download** text field specifies which *.nvi* file will be downloaded to the device's EEPROM.

The **Block Write Protect** radio button group allows you to specify whether to leave the block write protect value alone, or to set it to a specific value after the download operation completes.

IMPORTANT: If "*Do not modify block write protect*" is set, NVMD will *not* disable block write protect for download. It might execute without errors, even if this results in an invalid image or no change to the EEPROM.

The **Automatically unrestrict the device** radio button group controls what happens if a device is in restricted mode when the download operation begins: the software automatically unrestricts the device, or you are prompted for confirmation. Choosing not to unrestrict a restricted device before downloading causes the download operation to fail.

The **Additional Options** control group provides checkboxes and an editable text field for the following purposes:

- To disable the automatic size validation that takes place prior to downloading an image file to the device

CAUTION: The EEPROM size validation is enabled by default; only disable it if your device does not support the validation operation. The validation step ensures that the attached EEPROM is large enough to hold the image to be downloaded. Downloading an image that exceeds the available EEPROM size results in overflow and might render the device non-bootable.

- To specify whether the entire EEPROM is erased before downloading
- To allow the overwriting the manufacturing area during wipe and download operations

CAUTION: By default, the manufacturing area is not affected by either the wipe operation or the image download operation. Selecting the option to **Allow overwriting of manufacturing area** removes the write protection on the manufacturing area and allows the device-specific information stored there to be overwritten. Use of this option is not recommended in most cases as it results in the loss of important information, such as calibration data written at the time of the production of the device.

- To specify additional command line options that are passed to NVMD for use in the download operation; for a full list of the available options, refer to Table 25 on page 81

9.2.1.2.2 Connection Tab

The IDE is capable of communicating with a variety of development boards through a variety of communication devices. You can specify which protocol and communication device is used for a download operation in the Connection tab, shown in Figure 9.

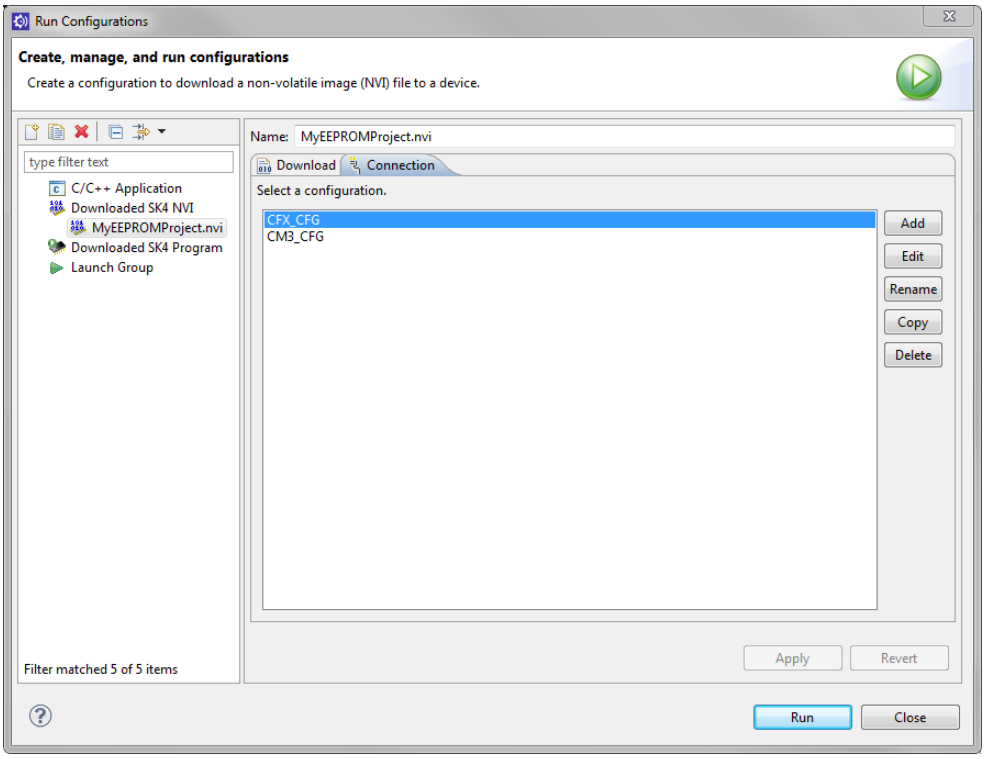


Figure 9. NVMD Launch Configuration Connection Tab

The Connection tab presents a list from which you can select a communication configuration for use by the debug session. You can also add, edit, and remove communication configurations from the list with buttons on the right side. More details on managing communication configurations are given in the *Integrated Development Environment User's Guide*.

IMPORTANT: Communication configurations are global settings. Changing or removing communication configurations affects all of the launch configurations that rely on them.

9.2.2 Usage

NVMD is a command line tool that can also be executed directly from a command prompt. The help (`-h`) option to NVMD provides details on all options and parameters, and they are described in Table 25. A simple command line for using the tool consists of the program name and an image file:

```
nvmd [options] image
```

The **`image`** file is a valid, non-volatile image (NVI) file that is the output of NVMM. For details on using NVMM, refer to Chapter 8, "Non-Volatile Memory Management" on page 57.

Table 25. NVMD Tool Options

Option	Name	Description
`-h`	Help	Passing this option causes NVMD to output a summary of the available options.
`-version`	Version	Passing this option causes NVMD to output the current version number of the executable file.
`-v`	Verbose Mode	By default, NVMD operates in quiet mode and only displays warnings or errors. To see more information about the image download process, `-v` can be passed to NVMD on the command line. In this case, NVMD will operate in verbose mode and will display information about the progress of the download.
`-u`	Unrestrict	If the device that NVMD is attempting to download the image to is in restricted mode, NVMD will abort the download attempt and fail by default. The `-u` option tells NVMD to automatically unrestrict the device and continue the download process instead.
`-f` **`range`**	Fill	Fills the device's non-volatile memory at the specified range with `0xFF` (by default). An alternate byte-value can be specified using the `-x` option. Ranges are specified using the format: **`start:`**`[`**`length`**`]` where **`start`** is a valid address in memory and the optional **`length`** argument is the length in bytes, separated by a colon. Values can be specified in decimal, octal or hexadecimal format. NOTE: if **`length`** is omitted, the area from **`start`** to the end of available memory will be filled. An explicit value for **`length`** *must* be specified if used in conjunction with the `-NoSizeCheck` option.
`-x` **`value`**	Fill Value	Specifies the value of the hex byte to be used in conjunction with the `-f` option. This option is ignored if `-f` is not specified.
`-w`	Wipe	Wipes the entire contents of the device's non-volatile memory before downloading by filling all of non-volatile memory with `0xFF`. This option is different from the partial system wipe that occurs as a side-effect of the Unrestrict option. NOTE: This option is *deprecated*. Use `-f 0x0000:` instead.

Table 25. NVMD Tool Options (Continued)

Option	Name	Description
`-b` **`setting`**	Clear and Set Block Write Protect	This option serves two purposes: first, it disables block write protect on the device if it is enabled (by default NVMD will leave the BWP setting as-is); secondly, it sets a particular block write protect setting upon completion. Valid block write protect values are: `0` or `"none"` - Disable block write protect `1` or `"upper 1/4"` - Block write protect upper 1/4 `2` or `"upper 1/2"` - Block write protect upper 1/2 `3` or `"all"` - Block write protect all NOTE: If `-b` is not defined, NVMD will *not* disable BWP for download and might execute without errors, even if this results in an invalid image, or no change to the EEPROM.
`-c` **`config`**	CTK Configuration	Tells NVMD to use the given CTK configuration to connect to the device. If this option is not supplied, NVMD will open a CTK configuration select dialog when it is called.
`-ForceOverwriteManufacturingArea`		Tells NVMD to write the block identified as the manufacturing area (node type “MftrArea”) to the device during the download. By default, a manufacturing area is never overwritten. NOTE: *Use extreme caution* when enabling this option as it results in the loss of all existing data in the manufacturing area, including important calibration values written during the production of the device.
`-NoSizeCheck`		Forces NVMD to skip validation of the device's non-volatile memory size prior to downloading the image. By default, the size of the attached non-volatile memory is always verified to ensure that the downloaded image will not exceed it. NOTE: Downloading an image which exceeds the physical size of the non-volatile memory on the device will result in overflow and may render the device non-bootable.

9.3 Non-Volatile Image Dump Tool

The Non-Volatile Image Dump (NVIDUMP) Tool is a command line tool that can be used to dump the raw data contained in a binary, non-volatile image (NVI) file to various other formats. The help (`-h`) option to NVIDUMP provides details on all options and parameters, and they are described in Table 26. A simple command line for using the tool consists of the program name and an image file:

```
nvidump [options] image
```

The **`image`** parameter is a valid, non-volatile image (NVI) file. Non-volatile image (NVI) files are created with the NVMM. For details on using NVMM, refer to Chapter 8, “Non-Volatile Memory Management” on page 57. The default output of NVIDUMP is an absolute-formatted dump of the data contained in the non-volatile image file. Additional formats are supported through command line options in Table 26.

Table 26. NVIDUMP Tool Options

Option	Name	Description
`-h`	Help	NVIDUMP outputs the tool version information and a summary of the available options.
`-v`	Verbose Mode	Use this option in conjunction with the `--vfs` option to include formatted data chunks in the output, along with the VFS information.
`-o` **`file`**	Output File	Capture the output in the given file. If this option is not specified, NVIDUMP displays the output in the console.
`-f` **`start:end`**	Fill Gaps	Fill any gaps in the data contained in the Intel HEX or Motorola SREC output file within the specified range with 0xFF (by default). An alternate byte-value can be specified using the `-x` option. Ranges are specified using the format **`start:end`** where **`start`** is a valid address less than or equal to the lowest address in the data file, and **`end`** is a valid address greater than or equal to the highest address, separated by a colon. Values can be specified in decimal, octal or hexadecimal format.
`-x` **`value`**	Fill Value	Specify the value of the hex byte to be used in conjunction with the `-f` option. This option is ignored if `-f` is not specified.
Options for Output Formats		
`--vfs`	VFS Format	Output the Virtual File System (VFS) data contained in the image file including general file system information and supported image features. By default, absolute data is displayed in the output.
`--intelhex`	Intel HEX Format	Output the data contained in the NVI file in Intel HEX format instead of the default absolute data format.
`--motorolasrec`	Motorola SREC (S19) Format	Output the data contained in the NVI file in Motorola SREC (S19) format instead of the default absolute data format.

APPENDIX A
File Formats

A.1 Introduction

CFCC uses two binary file formats, for the outputs of the assembly and linking stages, respectively. The executable files produced by the linker are in the executable and linking format (ELF), and the object files produced by the assembler are in a relocatable object file format.

A.2 Executable and Linking Format (ELF)

The file extension for ELF files is *.cfe*. ELF is a standard file format that can be interpreted from many publicly available documents. The *ELF for the ARM Architecture* document is especially relevant to the SK4 ELF Toolchain.

A.3 Relocatable Object File Format

The file extension for relocatable object files is *.o*. The assembler generates relocatable object files in a format called “elf32-little”, which is a 32-bit ELF file. You can get more information about it from publicly available documentation.

A.4 Non-Volatile Memory Image Format (.nvi)

The NVI file format is a non-proprietary, binary file format for storing non-volatile memory images once the layout has been processed. It stores data in a format that is quick and easy to extract for production, while maintaining sufficient metadata to read back the layout with no loss of information. The IDE provides a read-only editor, allowing the contents of the NVI file to be viewed, as well as a virtual representation of the values in non-volatile memory. NVMD can also be used with the `-v` option to output file information while the NVI file is being downloaded to the device’s EEPROM. You can dump the contents of NVI files to a variety of formats using the NVIDUMP tool; see Section 9.3, “Non-Volatile Image Dump Tool” on page 82 for more information.

You can create NVI files by using the IDE builder. The following is an outline of the current NVI file specification.

A.4.1 NVI File Structure

NVI files contain binary data encoding the complete layout of the EEPROM image as it will be downloaded to an attached device. The fundamental components of the file structure can be broken down into the following elements:

- Headers
- String Table
- VFS Node Chunks
- Data Chunks
- CRC Values

A.4.2 Headers

There are essentially two types of headers in a *.nvi* file: the main file header, which is a single entry found at the beginning of the file; and block headers, based on a generic template, which serve as the starting point for each block.

A.4.2.1 Main File Header

An NVI file must start with a proper file header. The header determines whether or not the file is a valid NVI file. It also serves as the starting point for any type of traversal or parsing of layout information. The structure of the file header is outlined in Table 27.

Table 27. Main File Header Structure (36 bytes)

Field Size	Field Name	Definition
byte[8]	MAGIC_NUMBER	Magic identifier for validation: actual value “SORA:SAI”.
int32	HEADER_SIZE	Size of the main header in bytes (36).
int8	VERSION_MAJOR	Major version for file specification. 0x00 for the current version.
int8	VERSION_MINOR	Minor version for file specification. 0x04 for the current version.
int16	BLOCK_COUNT	The number of blocks in this file.
int32	FEATURES	A set of flags designating features present in the file.
int16	TARGET_FS	Target file system.
int8	TARGET_FS_VER_MAJOR	Major version of the target file system.
int8	TARGET_FS_VER_MINOR	Minor version of the target file system.
int32	DESCRIPTION_ID	Pointer to a description string in string table.
int32	COPYRIGHT_ID	Pointer to a copyright message in string table.
int8	ATOMIC_UNIT	The size in bits of the atomic unit used in the file system of the stored image.
int8	BYTE_ORDER	The byte order for the stored image (0=big-endian, 1=little-endian).
int16	VFS_ROOT	Block pointer to the VFS root node. Null if and only if the VFS_EMBEDDED feature flag is not set.

A list of the currently supported features, which collectively contribute the features present in the file through a bitwise OR of their individual values, can be found in Table 28.

Table 28. Supported Feature Flags

Value	Feature Name	Definition
0x00000001	COMPLETE_RAW	Denotes a completely raw image containing only a single data block
0x00000002	HAS_KEYS	Specifies an image containing a key section to store shared keys for level 1 security (reserved for future use)
0x00000004	VFS_EMBEDDED	Indicates the presence of an embedded tree structure containing extended layout information in the image

A list of file systems currently supported by the NVI file format is included in Table 29.

Table 29. Supported File Systems

ID	File System Name	Definition
0x0004	SK4FS	The SK4 file system, used in Ezairo 7100.

A.4.2.2 Generic Block Header

While the main file header outlined in the section above is unique in the NVI file structure, it also shares similarities with the generic block header that serves as a template for every block in the file. For example, you can see that the `MAGIC_NUMBER` that begins the main file header is simply a specific application of the generic file header `BLOCK_TYPE` field. The structure of the generic block header is outlined in Table 30.

Table 30. Generic Block Header Structure (16 bytes)

Field Size	Field Name	Definition
byte[8]	BLOCK_TYPE	8-character string uniquely identifying the block type
int32	BLOCK_HEADER_SIZE	Total size of the block header (not including body)
int32	BLOCK_SIZE	Total size of the block (including the header and body). NOTE: Pad all blocks to a 32-bit boundary: (BLOCK_SIZE % 4 = 0).

Blocks are placed sequentially in the NVI file, one after another in increasing order, starting at 0. Block pointers, used as references between blocks, hold a 16-bit integer value specifying the unique number of the block to which they point. A block pointer of 0xFFFF represents a null block (i.e., no block). It is possible to iterate through the blocks in the NVI format by reading the generic block header and using the BLOCK_SIZE field to jump to the next block, and so on. The following sections describe some of the specific instances of blocks used in the NVI format.

A.4.3 String Table

The string table holds any human-readable strings referenced within the NVI file structure. For example, the strings can be used to denote names and attributes. This block is unique within the NVI file, occurring only once. In a sequential block listing, it is the first to follow the main file header that contains an actual body. The strings contained in the body of the block are encoded with the `UTF-8` format, terminated by the '`\0`' character. Null characters are used to pad the table to the 32-bit boundary.

NOTE: The information in the string table is useful when working with your NVI image in the IDE; however, it is not included in the information downloaded to the device.

The string table is preceded by a header based on the generic block header discussed above. The actual structure of the string table header is outlined in Table 31.

Table 31. String Table Block Header

Field Size	Field Name	Definition
byte[8]	BLOCK_TYPE	Actual value "StrTable"
int32	BLOCK_HEADER_SIZE	20 bytes
int32	BLOCK_SIZE	BLOCK_HEADER_SIZE + sizeof(string data) + padding
int32	STR_COUNT	The number of strings in the table.

The string pointers found in other blocks in the NVI file point to an offset from the beginning of the string data in the string table. Use the following offsets to calculate the absolute position of a string in the NVI file:

```
offset to start of string table header + BLOCK_HEADER_SIZE of string table header +
   offset to string in table data (string pointer value)
```

The value of 0xFFFFFFFF is a special value that refers to the null string.

A.4.4 VFS Node Chunks

If the main file header includes the VFS_EMBEDDED feature among the set flags encoded in the FEATURES field, then the VFS_ROOT field (also in the main header) will contain a block pointer to the first (root) VFS node

chunk. The VFS node chunks encode additional layout information including real and virtual parent-child relationships, attributes, human readable names and node typing, as well as the physical data chunks.

The structure of the VFS nodes is analogous to a tree, where the root node (following with the tree analogy) is the layout itself. Children of the layout can be virtual or real. Real elements contained in the VFS structure are always associated with a data chunk. VFS node chunks are ordered consecutively, following the string table in the NVI file. Each VFS node chunk starts with a header. The structure of the header is outlined in Table 32.

Table 32. VFS Node Chunk Header

Field Size	Field Name	Definition
byte[8]	BLOCK_TYPE	Actual value “VsfNode”
int32	BLOCK_HEADER_SIZE	36 bytes
int32	BLOCK_SIZE	BLOCK_HEADER_SIZE + (8 * ATTRIB_COUNT) + (2 * CHILD_COUNT) + padding
byte[8]	NODE_TYPE	8-character string denoting node type. Implementation is file system specific
int32	NODE_NAME	String pointer to a human-readable name of a block
int16	DATA_CHUNK	Block pointer to an associated data chunk; can be null (0xFFFF) for nodes with no data (virtual)
int16	DATA_SUBCHUNK	Reserved for future use; must be null (0x0000) in the current version
int16	ATTRIB_COUNT	The number of attributes stored in this node.
int16	CHILD_COUNT	The number of child nodes associated with this node.

Following the VFS node chunk header, the references to attributes and children are placed consecutively in their respective order. Attributes are stored as 32-bit string pointer to a key in the string table, followed by another 32-bit string pointer to the associated value. Child references take the form of a 16-bit block pointer indicating the associated child node.

NOTE: Names and attributes associated with the VFS and stored in the string table are generally derived from settings in the NVMM builder (e.g., application and section names from *.cfe* source files). When the information is not available, such as when an NVI image is created from uploading a device EEPROM, default names and attributes are generated.

A.4.5 Data Chunks

The data chunks contain the physical data that is ultimately transferred to the device EEPROM (or in the case of an NVI image created by uploading the contents of a device EEPROM, they hold the data which has been read.) Each data chunk is, as with the other blocks already discussed, preceded by a header based on the generic template. The specific structure of a data chunk header is outlined in Table 33.

Table 33. Data Chunk Header

Field Size	Field Name	Definition
byte[8]	BLOCK_TYPE	Actual value “DatChunk”
int32	BLOCK_HEADER_SIZE	32 bytes
int32	BLOCK_SIZE	BLOCK_HEADER_SIZE + CHUNK_SIZE + padding

Table 33. Data Chunk Header (Continued)

Field Size	Field Name	Definition
int16	NODE	Block pointer to associated VFS node; can be null (0xFFFF)
int16	SUB_CHUNK_COUNT	Reserved for future use. Must be 0x0000 in the current version.
int32	CHUNK_SOURCE	String pointer to source of block (e.g. application filename) in string table.
int32	CHUNK_SIZE	Size in bytes of the raw data contained in body to follow.
int32	DEST_ADDRESS	The destination address (in non-volatile memory) to which this chunk of data is to be copied.

The data that follows the header is stored as raw bytes to be transferred to the device during a download operation.

A.4.6 CRC Values

Note that in the interest of data validation and integrity, the NVI specification includes a CRC value calculated over the file as a whole. The file CRC is calculated as a 32-bit integer and written to the last 4 bytes of the file. Any conflicts detected in the CRC verification are generally interpreted as file errors.

ON Semiconductor and ON are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

M-20804-010